

THE PATENT MINER
CJ VINES
CHRISTIAN CASTEEL
KOSI EZEWUDO

FINAL REPORT

REVISION – 2
27 April 2026

Table of Contents

Concept of Operations	1
Functional System Requirements	15
Interface Control Document.....	29
403 Execution and Validation Plan.....	40
403 Subsystem Reports	43
ECEN 404 Final System Report	61
GitHub Repository:.....	61

THE PATENT MINER
CJ VINES
CHRISTIAN CASTEEL
KOSI EZEWUDO

CONCEPT OF OPERATIONS

CONCEPT OF OPERATIONS FOR THE PATENT MINER

TEAM 17

APPROVED BY:

CJ Vines	12/6/25
----------	---------

Prof. Nowka	12/6/25
-------------	---------

Fahrettin Ay	12/6/25
--------------	---------

Change Record

Rev.	Date	Originator	Approvals	Description
-	9/17/2025	CJ Vines		Draft Release
-	12/6/2025	CJ Vines		Draft Update
-	4/25/2026	CJ Vines		Final Update

Table of Contents

1. Executive Summary	6
2. Introduction	6
2.1. Background	6
2.2. Overview	7
<i>Figure 1. Process flow from USPTO database to vector storage.....</i>	<i>7</i>
<i>Figure 2. Swim diagram of query pipeline.....</i>	<i>8</i>
2.3. Referenced Documents and Standards	9
3. Operating Concept.....	9
3.1. Scope	9
3.2. Operational Description and Constraints	9
3.3. System Description.....	10
3.3.1. Preprocessing Subsystem	10
3.3.3. RAG subsystem	11
3.3.4. User Interface Subsystem	11
3.4. Modes of Operation	12
3.4.1. RAG mode.....	12
3.4.2. Simple Search Mode	12
3.5. Users	13
3.6. Support.....	Error! Bookmark not defined.
4. Scenario(s)	13
4.1. Find Prior Art	13
4.2. Patent Novelty	13
4.3. Patent Research.....	13
4.4. Summary of Proposed Improvements	14
4.5. Disadvantages and Limitations	14
4.6. Alternatives.....	14
4.7. Impact	14

List of Figures

Figure 1. Process flow from USPTO database to vector storage	7
Figure 2. Swim diagram of query pipeline	8
Figure 3. Process flow from query to retrieval	10

1. Executive Summary

The United States Patent and Trademark Office (USPTO) database currently uses a keyword search method, which fails to capture contextual meaning. Current searches often return numerous results not relevant to the user. The Patent Miner addresses this by utilizing ColBERT and MUVERA to embed semantic meaning into patent documents. The system downloads patents from the USPTO database, embeds patent text, and stores the resulting vectors in a vector database for retrieval. The vector database will be accessible through a web API (Application Programming Interface) and integrated into RAG (Retrieval Augmented Generation) pipelines for LLMs (Large Language Models). This way, if a user were to request an answer to a question, the LLM can provide a more contextualized answer.

2. Introduction

Finding prior art for a patent can be difficult. The word “China” can link to either the country China or porcelain glassware. Parsing through documents for words without context can lead to documents with no relation to the actual meaning of the user’s query. This document introduces Patent Miner as a scalable search index that can interpret queries and give contextualized, relevant results to the user while providing a RAG summary. The Patent Miner will have a front-end for retrieval of patents and context window for interactions with the LLM utilized in the RAG pipeline.

2.1. Background

Currently, the USPTO database provides only a basic keyword search function that returns every instance of the queried terms across millions of documents. Patent lawyers and litigation specialists are often costly, leading to significant legal fees for companies that keep them on retainer. When searches are ineffective, serious legal and financial consequences can occur if prior art is overlooked or if an invalid claim is made against a patent.

The Patent Miner intends to address this issue by utilizing multivector embeddings, which preserve contextual information by representing each token relative to its surrounding words. These token-level vectors will then be compressed into a FDE (Fixed Dimensional Encoding) through projecting all vectors in a claim onto random gaussian vectors. The tool will have filters for document id, date, and dependent claims for the user to enable and further tune their search. An LLM will be integrated to handle user queries, summarize retrieved documents, and support the polished front-end interface.

2.2. Overview

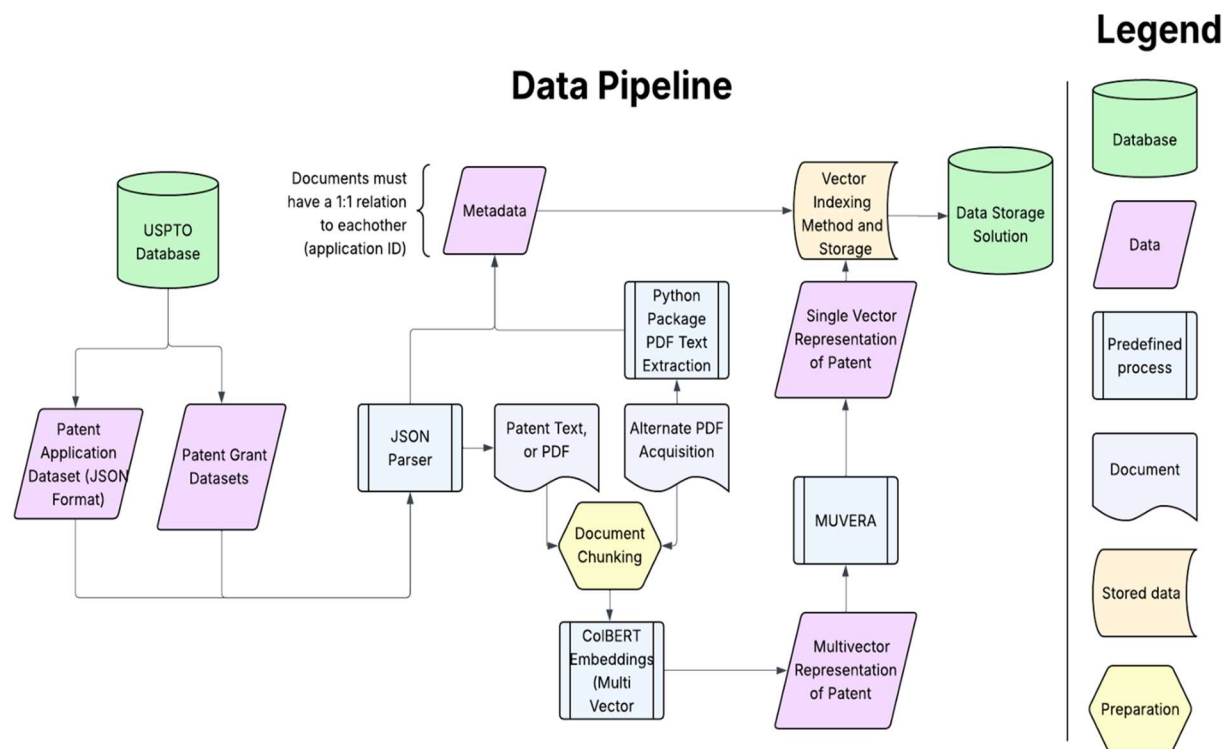


Figure 1. Process flow from USPTO database to vector storage

The system will accept patent grant XML files from the USPTO bulk dataset API and extract metadata from each document before embedding claims. The resulting corpus will be stored in Weaviate where it can be retrieved by the front-end website. This vector database will be scaled up to fill the Data Storage Solution as much as possible so that a large breadth of patents will be available.

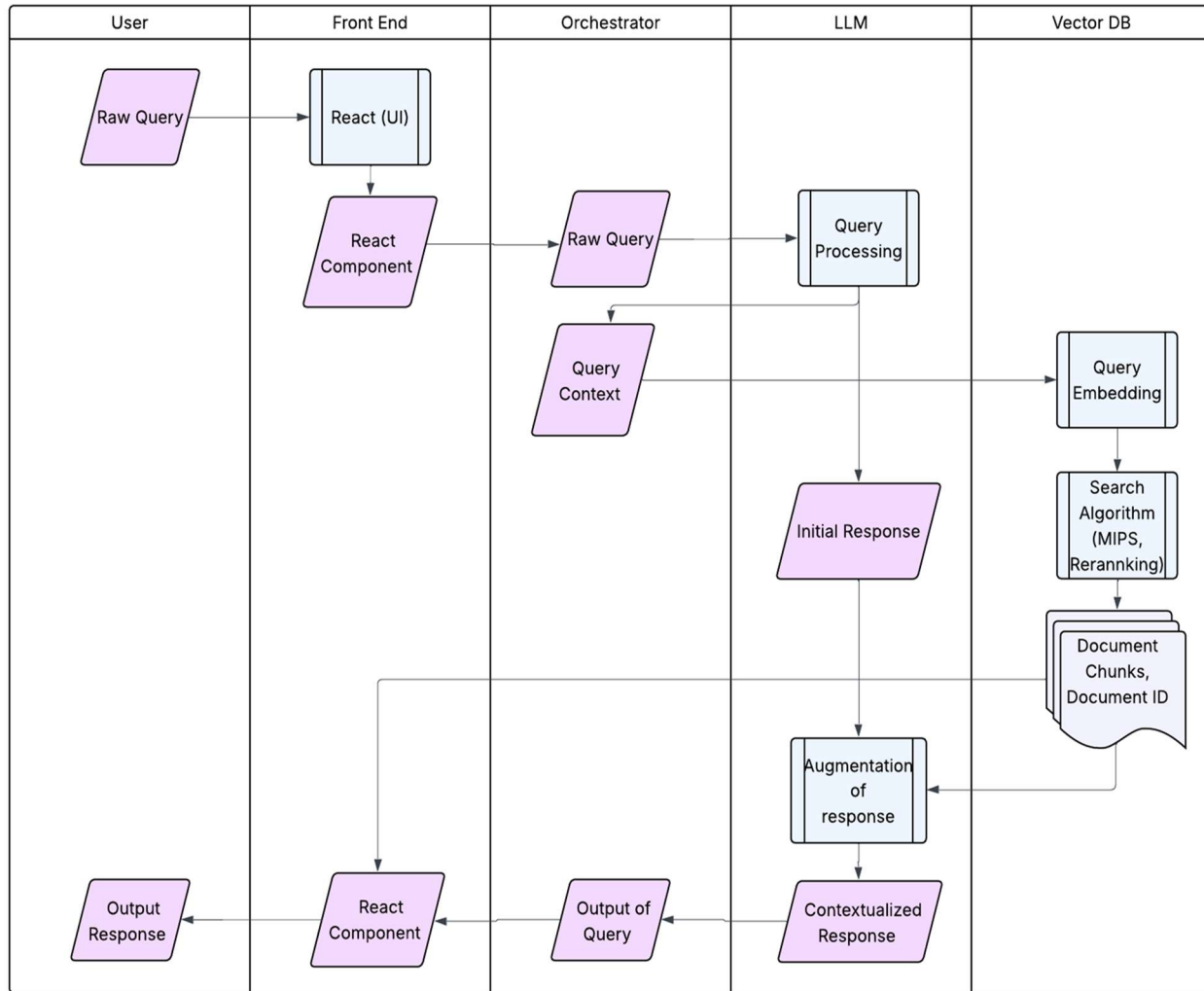


Figure 2. Swim diagram of query pipeline

This front-end website will be fully integrated with a RAG pipeline for summarization of patents, as well as more accurate responses to questions. An orchestrator will be used to access a large language model and control the process flow depending on the type of input. An API will be implemented that interacts with both the orchestrator as well as the backend vector database, wrapped around a React frontend. The system will perform a semantic search and order the results from most relevant to least relevant.

2.3. Referenced Documents and Standards

1. MUVERA: Making multi-vector retrieval as fast as single-vector search. (n.d.). <https://research.google/blog/muvera-making-multi-vector-retrieval-as-fast-as-single->
2. Team, M. (2024, October 3). RAG Pipeline Diagram: How to augment LLMs with your data. RAG Pipeline Diagram: How to Augment LLMs With Your Data. <https://www.multimodal.dev/post/rag-pipeline-diagram>
3. United States Patent and Trademark Office. (n.d.). APIs. Retrieved September 17, 2025, from <https://data.uspto.gov/apis/>

3. Operating Concept

3.1. Scope

The Patent Miner is a system focused on patent analysis, attack, and defense by quickly and accurately retrieving prior art. The Patent Miner aims to ease the task of patent search and retrieval from the USPTO database. The mining tool will be designed to only operate on documents from 2001-present. The successful completion of this project will lay the foundation for further extension of the Patent Miner to operate on patent applications prior to 2001.

The Patent Miner required modern software resources such as embedding models, large language models such as ChatGPT, vector databases for information storage, large-scale computing resources for extensive computation, and web development toolkits. The project timeline is ten months, at the end of which the goal is to deliver proof of concept.

3.2. Operational Description and Constraints

The Patent Miner is intended for use in the academic, industrial and law fields. The system will feature a user interface which connects through an API to the backend mechanism where all patent search and retrieval shall take place. The USPTO database is updated weekly; therefore, the mining tool algorithm is designed to update accordingly. Given the above description, the mining tool and its users are constrained by the following factors:

- The POC system that is built must be small enough to be run on a personal computer for testing purposes before being scaled up.
- Sufficient data storage (5 Tb+) for old, current and new patents is needed. The budget for the project is \$375, which limits the quantity of storage that can be used during the developmental phase.
- Application interface access to the U.S patent website for patent retrieval is needed.
- one patent can only be downloaded every 2 seconds from the USPTO database, and each file can only be downloaded 3 times a year.
- Each bulk dataset API call may only be made 20 times on the same set.
- Once the POC has reached it's size constraint on the personal computer, the system must be migrated to a Data Storage Solution

3.3. System Description

The mining tool will consist of four primary subsystems: Preprocessing subsystem, Patent embedding subsystem, RAG subsystem, and User Interface subsystem. The following section gives a system overview of each category.

3.3.1. Preprocessing Subsystem

The download process involves usage of the USPTO Bulk Dataset API. One of the products of the Bulk Dataset API is the PTGRDT, which contains all of the granted patents from the past week. Patents are available in XML format, which is organized in a node tree hierarchy. These XMLs include all relevant metadata, abstract, description, and claims of the patent. The abstract, description and claims will be separated into what are known as “chunks”. These chunks are separated by newline, or by a maximum token count, whichever comes first. After this process, the chunks are then aggregated in batches to be embedded to save memory. Metadata for the patent will be packaged with the chunks so that when a chunk is retrieved, the metadata comes along with it.

3.3.2. Patent Embedding subsystem

This subsystem encodes the text contained in each patent into numerical vector representations using modern information retrieval methods. Patent documents are divided into smaller text chunks, and each chunk is embedded using the ColBERT (Contextual Late Interaction over BERT) algorithm. ColBERT produces multiple token-level vectors for each chunk, allowing the system to capture fine-grained semantic meaning rather than relying on single document-level embedding.

The resulting token embeddings for each chunk are stored in a vector database, where they can be efficiently indexed and retrieved using MUVIRA. MUVIRA reduces the matrix of token level vectors into a single dense vector to improve retrieval time at a small cost of accuracy. Relevant chunks are reranked using the original ColBERT vectors to avoid the loss of accuracy. During a search, the user’s query is also embedded using ColBERT, then MUVIRA, and vector similarity scoring is used to identify the most relevant chunks across the corpus. These top-ranked chunks are returned to the user interface and may also be passed to an LLM for summarization or contextual question answering.

3.3.3. Orchestrator subsystem

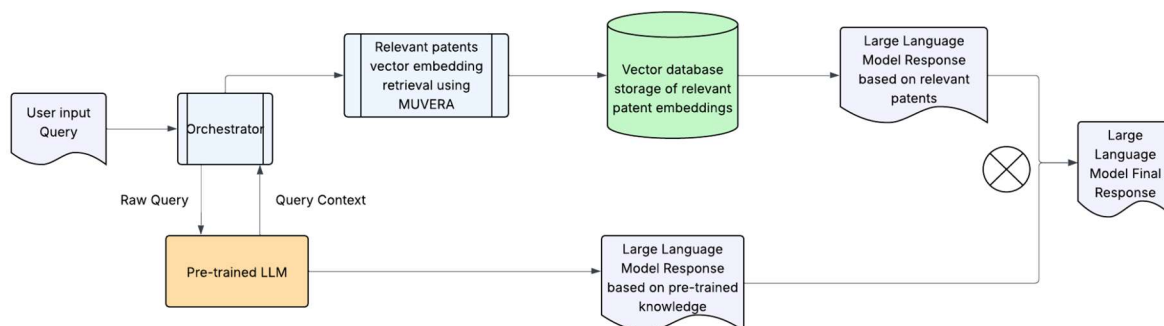


Figure 3. Process flow from query to retrieval

The orchestrator subsystem combines information retrieval with text generation. The orchestrator subsystem will efficiently allow for the usage of retrieved documents in answering further questions by a user. The orchestrator will be built in the Langsmith ecosystem and feature pre-trained LLMs. A pre-trained LLM in the RAG pipeline uses the retrieved documents from MUVIRA as external knowledge in addition to its own internal knowledge to give contextually relevant responses.

The orchestrator is implemented to allow for an enhanced user-friendly nature to the Patent Miner and serve as an improvement on current patent search and retrieval tools such as Google Patents.

3.3.4. User Interface Subsystem

The User-Interface subsystem for the patent miner is implemented through a Web Interface. The web interface will be created using React and Node.js-common front-end web design libraries. The front end will communicate with the backend and RAG through an API called FAST API. FAST API utilizes Pydantic to validate that data structures being passed from one language to another do not lose any information. Generated RAG responses and patent results will be returned to the front end through FAST API.

3.4. Modes of Operation

The entire system has 2 modes of operation: patent search mode and claim search mode. Brief descriptions of each are given below

3.4.1. Patent Search Mode

In patent search mode, the user submits a query and filters while the patent search mode is active. Patent search mode is a parameter routed through FAST API to the orchestrator and backend subsystems. When active, Weaviate will retrieve more candidates to be reranked. Once reranking is completed, each patent will be represented by its highest-ranking claim, all other claims are eliminated from search results. Each result in this search mode is a unique patent.

3.4.2. Claim Search Mode

In claim search mode, the user submits a query and filters while the claim search mode is active. Claim search mode is a parameter routed through FAST API to the orchestrator and backend subsystems. This is the default mode of operations for the system. The backend will conduct normal retrieval and return claim results.

3.5. Users

The Patent Miner will primarily be used by lawyers and litigation specialists for the purpose of claim defense. With either basic or detailed queries, the Patent Miner will locate patents with matching claims. Additionally, researchers and inventors are considered. The Patent Miner can serve as a research tool or can determine the originality of a given patent's claims. This will also be achieved via a simple user query, and casual users have access to all these features as well.

4. Scenario(s)

4.1. Find Prior Art

Upon user query, the Patent Miner will find patents with matching context and claims. Once these patents are identified, the patent miner checks the priority date and determines the merit of a claim. This allows the user to counter claims of prior art and protect their intellectual property from external parties. Conversely, this allows the user to gauge the validity of their own patents compared to existing ones.

4.2. Patent Litigation

To invalidate a claim infringement lawsuit, Patent Miner will find similar patents using its embedded database, utilizing date filtering to ensure that patents could potentially be prior art. Since infringement lawsuits can be costly, this scenario could potentially save the user thousands to millions of dollars.

4.3. Patent Research

The Patent Miner can serve as a research tool for patents within the USPTO database. The user can propose the idea of a certain invention, and the Patent Miner will search for patents relating to the query. Upon user request, the Patent Miner will summarize and explain specific patents.

Analysis

4.4. Summary of Proposed Improvements

- The system shall retrieve relevant documents with semantic similarity score, with ColBERT and MUVERA being used to embed and retrieve.
- The system shall rerank documents with similarity score, with ColBERT MaxSim scoring being the chosen method
- The system shall utilize an outside LLM that will allow for not only retrieval, but summarization of patents, with ChatGPT 5.2 being the chosen LLM.

4.5. Disadvantages and Limitations

- The full USPTO database will necessitate a very large vector database.
- Lack of in-house LLM means API calls will bottleneck search speeds.
- The home PC being used for embedding means that embedding will also bottleneck the embedding pipeline

4.6. Alternatives

Alternative solutions would be to use a single vector system. This would enable faster search and better storage at the cost of some semantic information. The MUVERA algorithm can compress multivector embeddings to single vector while preserving contextual meaning. The current truncation system on chunks exceeding the token limit can have a semantic chunker implemented. This would prevent patent text from being truncated, and ensure that chunked sections retain as much contextual meaning as possible.

4.7. Impact

- The system will streamline the process of finding prior art, saving time from individuals creating duplicate patents, help smaller companies research patent information, and reduce frivolous lawsuits as a result of claims.
- The system will provide this technology to smaller companies and individuals. All existing patent mining technology is exclusive to larger companies.
- Users can be always up to date with the latest patents as patent applications will be added daily, making it easier to keep up with trends in technology.

THE PATENT MINER
CJ VINES
CHRISTIAN CASTEEL
KOSI EZEWUDO

FUNCTIONAL SYSTEM REQUIREMENTS

REVISION – 2
27 April 2026

FUNCTIONAL SYSTEM REQUIREMENTS FOR THE PATENT MINER

APPROVED BY:

CJ Vines

12/06/25

Prof. Nowka

12/06/25

Fahrettin Ay

12/06/25

Change Record

Rev	Date	Originator	Approvals	Description
.				
-	10/04/2024	CJ Vines		Draft Release
-	12/6/2025	CJ Vines		Draft Update
-	4/25/2026	CJ Vines		Final Update

Table of Contents

1. Introduction	21
1.1. Purpose and Scope	21
1.2. Responsibility and Change Authority	22
2. Applicable and Reference Documents	22
2.1. Applicable Documents	22
2.2. Reference Documents	23
2.3. Order of Precedence	23
3. Requirements	24
3.1. System Definition	24
3.2. Characteristics	24
3.2.1. Functional / Performance Requirements	24
3.2.1.1. Preprocessing Determinism Requirements	24
3.2.1.2. Input Requirements	25
3.2.1.3. Query Preprocessing	25
3.2.1.4. Operational Mode Requirements	25
3.2.1.5. Retrieval Requirements	25
3.2.1.6. Search Results Requirements	25
3.2.1.7. Citation Reporting Requirements	26
3.2.1.8. Server Timeout	26
3.2.1.9. LLM Preprocessing Failure	Error! Bookmark not defined.
4. Support Requirements	27
4.1. Internet Device	27
4.2. User Support	Error! Bookmark not defined.
4.3. Installation and Deployment	27
4.4. Data Maintenance	27
Appendix A: Acronyms and Abbreviations	28

List of Tables

<i>Table 1.</i> Subsystems and Associated Subsystem Heads	21
---	----

List of Figures

<i>Figure 1.</i> Flow Chart	20
<i>Figure 2.</i> Block Diagram of System	23

1. Introduction

1.1. Purpose and Scope

The purpose of The Patent Miner is to enhance search and retrieval of patents from the United States Patent and Trademark Office (USPTO) database. The USPTO database uses a basic keyword search to return results, which tends to yield numerous irrelevant results. The Patent Miner addresses this issue with semantic search, which will utilize Large Language Models (LLMs) and multi-vector embeddings to extract meaning from patents. This will enable users to receive more accurate results when searching. The Patent Miner will operate entirely through software, and as a result there is no need for physical hardware or environmental testing.

The frontend UI subsystem will take in user input and present feedback on the relevancy of the search via LLM response. The Retrieval Augmented Generation (RAG) subsystem will embed the user query using the embedding pipeline subsystem and retrieve patents from the embedded patent database. The embedding pipeline will take patents from the USPTO database as well as user queries, embed them as multi-vector representations, and store them in a database. The end goal is to accelerate the work of researchers, inventors, and litigation specialists with a robust contextual search function contained in highly accessible software.

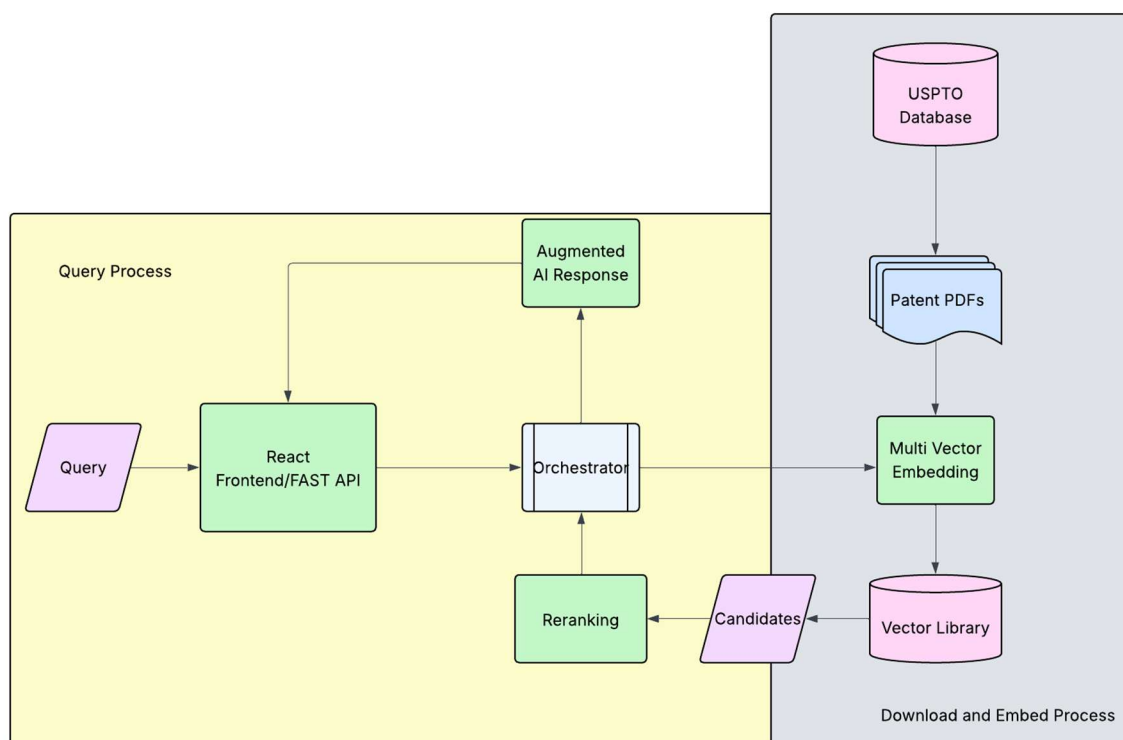


Figure 1. Flow Chart

1.2. Responsibility and Change Authority

Ultimately it is the duty of the head of each subsystem, as well as the project manager, to ensure system requirements are achieved in a timely manner. All individuals contributing to the project can make changes to the requirements of the system, but only after discussion with other members of the system to update requirements based on new information related to capabilities of the system, shortfalls of predefined processes being realized, or other issues met along the process in development of the system. Also, Dr. Kevin Nowka and other members of the faculty teaching 403 have authorization to make changes in the requirements of the system as they see fit.

Subsystem	Head
Embedding Pipeline	CJ Vines
Frontend/API	Christian Casteel
RAG/Orchestration	Kosi Ezewudo

Table 1. Subsystems and Associated Subsystem Heads

2. Applicable and Reference Documents

2.1. Applicable Documents

The following documents, of the exact issue and revision shown, form a part of this specification to the extent specified herein:

Document Number	Revision/Release Date	Document Title
PM1-CONOPS	9/17/25	Patent Miner Concept of Operations
PM1-ICD	10/4/25	Patent Miner Interface Control Document
N/A	8/25/25	ECEN 403 Introduction Slides

2.2. Reference Documents

The following documents are reference documents utilized in the development of this specification. These documents do not form a part of this specification and are not controlled by their reference herein:

Document Number	Revision/Release Date	Document Title
N/A	4/18/22	HuggingFace Tokenizers, https://huggingface.co/docs/tokenizers/index
N/A	9/29/25	HuggingFace Transformers, https://huggingface.co/docs/transformers/index
N/A	8/28/25	LangChain Documentation, https://python.langchain.com/docs/introduction/

2.3. Order of Precedence

In the event of a conflict between the text of this specification and an applicable document cited herein, the text of this specification takes precedence without any exceptions. Followed by that, the Concept of Operations will take precedence, followed by the Interface Control Document, and finally by the referenced documents and standards

3. Requirements

3.1. System Definition

The Patent Miner is an end-to-end, self-updating vector library of patents connected to an outside Large Language Model (LLM) and front end that allows a user to retrieve contextual information about patents. The embedding/preprocessing subsystems are the first of four subsystems, which utilizes CoBERT to create a multivector representation of each patent, MUVERA to condense the multivector representation into a single dense vector, Weaviate for storage of vectors and metadata, and a search function that utilizes the HSNW algorithm, allowing for accurate retrieval of contextual information. The vector library is connected to the orchestrator subsystem, which acts as the control flow of the query intake and response generation process. Finally, the frontend acts as an outward facing user interface which will teach the user how to use the tool, as well as generate links to full texts of patents and other relevant information.

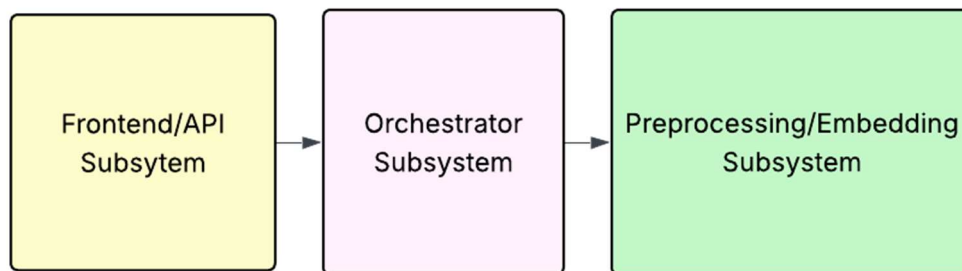


Figure 2. Block Diagram of System

In Figure 2, each subsystem is shown in the order in which they interconnect. The frontend subsystem will connect to the Orchestrator subsystem through an API that allows the Orchestrator and Preprocessing/Embedding subsystem to be hosted on a different machine than the Front End itself. The Front End will be a dynamic page with a hero and search bar, with a results page. FAST API will be used to limit connection to the back end as a cybersecurity measure.

3.2. Characteristics

3.2.1. Functional / Performance Requirements

3.2.1.1. Preprocessing Determinism Requirements

The USPTO Bulk Datasets offers full text patent grants in both PDF and XML. Full text patents with images shall be downloaded and parsed as XMLs. The download pipeline shall have lossless extraction of text and relevant metadata. The preprocessing subsystem shall not modify any data during downloading and extraction of any data.

Rationale: Patent Grant XMLs contain metadata for the patent at application time. During the patent review process, new CPC codes can be added to a patent to further define its subject matter.

3.2.1.2. Input Requirements

The frontend shall take in a query as a string and relevant filters to the search. This functionality will be validated by API tests, where a query is entered and a JSON in the relevant schema comes out.

Rationale: This is a baseline requirement for a RAG pipeline, someone needs to be able to ask a question about the corpus

3.2.1.3. Query Preprocessing

The query shall be accepted by the Orchestrator in JSON format, and an LLM will preprocess the query for any user errors, as well as possibly rewriting the query to maintain the comments.

Rationale: Requests could flood the server that have poor grammar, spelling, etc. that cause issues with search and bog down the experience for everyone else. Also gives time for jobs to be batched in a pipeline

3.2.1.4. Operational Mode Requirements

The system shall operate in 2 modes, patent search mode, and claim search mode. Both search modes will allow the user to enter queries and submit filters. Patent search mode will return entire patents as results. Claim search mode will return claim results.

Rationale: For more broad searches, such as for research purposes, the patent mode may be desirable for more breadth. For litigation purposes such as claim reconstruction, the claim search mode may be more desirable to see multiple close results.

3.2.1.5. Retrieval Requirements

The Retrieval system shall return the top-k requested documents in less than 2 minutes using MUVERA for candidate retrieval and ColBERTv2 for reranking. The documents shall be embedded asynchronously, and the query is embedded at the time of the query. These documents shall be ranked with MAXSIM, with a target Normalized Discounted Cumulative Gain (NDCG) ≥ 0.4 , recall@k ≥ 0.8 , and Mean Reciprocal Rank (MRR) ≥ 0.4 . Metadata shall be stored alongside vectors for efficient filtering during the retrieval stage.

Rationale: This is a requirement is a standard metric of search functions and allows for the grading of how relevant the results are to the query, offering a baseline for how useful the tool ought to be with the given operating conditions

3.2.1.6. Search Results Requirements

Each result shall include all metadata extracted during parsing, including document ID, application number, title, claims, applicant, grant date, Cooperative Patent Classification (CPC), and source URLs for the full text of the patents on Google Patents.

Rationale: Guided by the schema of the USPTO File Wrapper API, this is the host of information that is offered to the API accessor and shall be included for the user of The Patent Miner as relevant information.

3.2.1.7. Citation Reporting Requirements

When search results are displayed to the user, all results shall come packaged with all relevant metadata extracted during preprocessing. These results shall be placed on a sidebar with relevant hyperlinks to the full text on Google Patents.

Rationale: Displaying metadata of the patent to the user allows the user to verify the RAG response is accurate and also allows the user to continue independent research seamlessly without searching for the full text of the patents themselves.

3.2.1.8. Server Timeout

If a connection cannot be established after 10 tries with exponential backoff, the front end shall return an error message and ask the user to try again later.

Rationale: This keeps the front end from overloading a back end that is bogged down by requests

3.2.1.9. Response Generation Quality and Latency Requirements

The response generated to the user must have $\geq .8$ Faithfulness score and benchmark Latency results.

Rationale: The response generated by the LLM within the Orchestrator must be factually consistent with the retrieved context to prevent hallucination. In addition – response time should be on par with modern RAG systems to prevent long search times.

4. Support Requirements

4.1. Internet Device

The system will provide a web interface that requires an internet-connected device for access.

Rationale: Without internet access, a user cannot reach the remote API.

4.2. Installation and Deployment

The system shall store all production source code in a GitHub repository. The system shall provide users with access to update release notes documenting bug fixes and feature updates.

Rationale: GitHub is standard for version management. It provides clean updates of code and cross collaboration across subsystems. The release notes keep users informed.

4.3. Data Maintenance

The system shall query the USPTO Bulk Data API once per hour every Tuesday to identify newly released patents. The system shall update its internal database with any new or modified patents detected in the USPTO weekly release.

Rationale: The USPTO Bulk Dataset product PTRGDT is released every Tuesday, so any new patents will need to be added on that day.

Appendix A: Acronyms and Abbreviations

USPTO	United States Patent and Trademark Office
LLM	Large Language Model
CPC	Cooperative Patent Classification
API	Application Programming Interface
RAG	Retrieval-Augmented Generation
MRR	Mean Reciprocal Rank
NDCG	Normalized Discounted Cumulative Gain

**THE PATENT MINER
CJ VINES
CHRISTIAN CASTEEL
KOSI EZEWUDO**

INTERFACE CONTROL DOCUMENT

REVISION – 2
27 April 2026

INTERFACE CONTROL DOCUMENT FOR THE PATENT MINER

APPROVED BY:

CJ Vines

12/06/25

Prof. Nowka

12/06/25

Fahrettin Ay

12/06/25

Change Record

Rev	Date	Originator	Approvals	Description
.				
-	10/03/25	CJ Vines		Draft Release
-	12/6/2025	CJ Vines		Draft Update
-	4/25/2026	CJ Vines		Final Update

Table of Contents

1. Overview	35
2. References and Definitions.....	35
2.1. References	35
3. Patent Retrieval and Storage Interface	35
3.1. Patent Retrieval	35
4. Patent Storage - Orchestrator Interface	36
5. Orchestrator - LLM Interface	37
6. Frontend - Backend Interface	38
6.1. Incoming Requests	38
6.2. Outgoing Requests	39

List of Tables

<i>Table 1.</i> Description of USPTO Database.....	34
<i>Table 2.</i> Description of Vector Database.....	35
<i>Table 3.</i> Response codes that could be produced by the API.....	36
<i>Table 4.</i> JSON Schema of GET request.....	37
<i>Table 5.</i> JSON schema of response.....	37

List of Figures

Figure 1. Diagram of communication.....	36
---	----

1. Overview

The Patent Miner project proposes a search and retrieval tool for patents stored within the United States Patent Trademark Office (USPTO) Database. The Mining tool will consist of four primary interfaces: Patent Retrieval - Storage Interface, Patent Storage - Orchestrator Interface, Orchestrator - LLM interface, Frontend - Backend Interface. This document will cover the operating principle for each interface of the document, its components and how each interface connects with the next interface.

2. References and Definitions

2.1. References

Refer to Section 2.2 of the Functional System Requirements document

3. Patent Retrieval and Storage Interface

This interface is primarily made up of patent access and patent retrieval. The patents will primarily be accessed from the USPTO Database and stored in a vector database. The following sections describe the characteristics of each interface.

3.1. Patent Retrieval

Database	Characteristics	Size	Form of Patent Available	Form of Patent Access	Format
USPTO Database	Only 5 files per 10 seconds	All applications from 2001-2025, all grants from 1976-2025, 4.5+ TB	Bulk Datasets Delta Datasets	Bulk Datasets API	XML

Table 1. Description of USPTO Database

Following the description in *Table 1*, the current US patent database contains more than 4.5 TBs of patent applications and granted patents. The patents are available in either the bulk dataset form or delta dataset form. The bulk dataset form of the patents contains patents stored over defined periods and the delta dataset consists of both the bulk and daily update of the patent storage repository i.e., including both patent application and grants.

The patents can be accessed through one of many APIs that are connected to three storage repositories made of patent application bibliographies, raw public bulk data, and final agency petition decisions.

3.2. Patent Storage

Characteristics	Database	Size	Stored	Access
Description	Local Weaviate database (OLYPUS cluster)	920 GB	Token-level ColBERT embeddings, MUVIRA single dense vector embeddings, metadata, and patent chunk text	REST API

Table 2. Description of Vector Database

Table 2 provides an overview of the storage architecture for embedded patent data. After patents are downloaded and processed, each document is divided into chunks and embedded using ColBERT. These chunk embeddings, along with their associated metadata and extracted text, are first converted into MUVIRA single dense vector embeddings, then stored in a Weaviate instance on the OLYMPUS cluster at Texas A&M. The database occupies 920 GB. Weaviate automatically indexes the stored embeddings, enabling efficient vector similarity search and metadata-based filtering during retrieval.

4. Patent Storage - Orchestrator Interface

The Patent storage - Orchestrator interface is primarily responsible for handling processes related to using a user's input to retrieve contextually relevant stored patents. The AI Orchestrator to be used is LangChain. LangChain will enable efficient modularization of processes that ensure a user's input query is in a format that can be used to search for relevant patents within the local database. The user's query will be processed and embedded as a vector, then used to search for contextually relevant patents through the index of embedded patent vectors.

5. Orchestrator - LLM Interface

The Orchestrator - LLM interface will primarily be responsible for using the retrieved patents from the local database to generate a response to the user. LangChain - the orchestrator being used, will augment the LLM's original response to the user's query with context from patents relevant to the query.

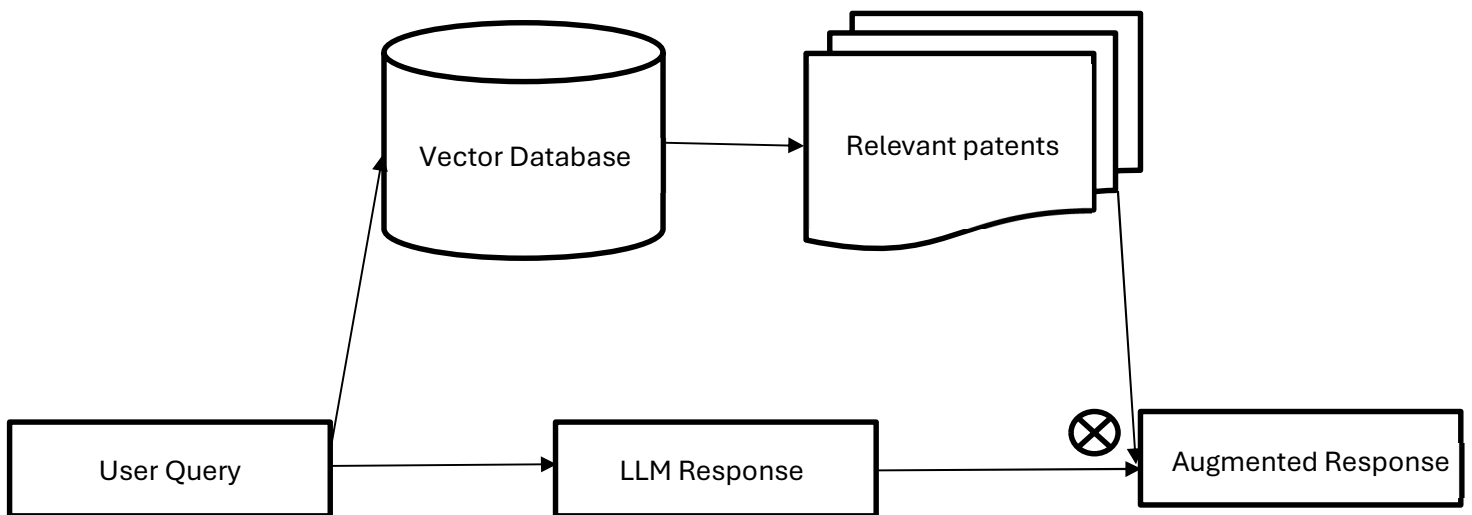


Figure 1. Diagram of communication between Patent Storage-Orchestrator-LLM

6. Frontend - Backend Interface

The Frontend - Backend of the patent mining tool will communicate through a REST API. FAST API will be used in order to match the base language of the backend. The API will serve primarily as a bridge between the user's input and the backend patent search and retrieval algorithm. The API will resolve GET and POST requests in the JSON data format through an HTTP protocol.

6.1. Incoming Requests

The Frontend will send a GET request to the backend. If the API is online, it will return a response code based on the result of the action being taken. Response codes include:

Response Code	Meaning
200 OK	System is behaving as intended
422 Unprocessable Entity	Failure of incoming data validation
404 Not Found	Unknown Route
405	Correct Route, but wrong type of request
500 Internal Server Error	Catchall for leftover errors

Table 3. Response codes that could be produced by the API

In order to make a request, below is the response schema required for operation and validated by Pydantic:

Variable Name	Type	Use
q	str	Contains the user query
k	int	The number of results to use in augmentation
k_extra	int	The number of extra results to report to user
rag	boolean	Determines RAG vs direct retrieval
search_scope	str	Determines claim or patent search mode

Table 4. JSON Schema of GET request

6.2. Outgoing Requests

After retrieval, augmentation and generation, the Orchestrator will provide Fast API with a response. Required output is listed below:

Variable Name	Type	Use
Query	str	Contains the user query
Total	int	The number of results
Page	int	The number of pages search results are grouped into
page_size	int	The number of results per page
Top	List	The retrieved chunks that are given to the LLM
cited_items	List	Items that are cited for the sidebar
other_items	List	Items that are not cited, but will be included in the sidebar
Mode	str	Contains information on what kind of path was taken (RAG or simple search)
Answer	str	Contains the actual RAG response

Table 5. JSON schema of response

THE PATENT MINER
CJ VINES
CHRISTIAN CASTEEL
KOSI EZEWUDO

403 EXECUTION AND VALIDATION PLAN

REVISION – 2
27 April 2026

Execution Plan for The Patent Miner

	09/01/25	09/08/25	09/15/25	09/22/25	09/29/25	10/06/25	10/13/25	10/20/25	10/27/25	11/03/25	11/10/25	11/17/25	11/24/25	12/01/25	Date		
Sprint 1															09/22/25		
Contact Sponsor															09/05/25		
Embedding Research															09/22/25		
LangChain Research															10/20/25	Not Started	
Frontend UI Research															09/22/25	Behind Schedule	
USPTO Reasearch															10/22/25	In Progress	
Sprint 2															10/20/25	Complete	
Development of Homepage/API															09/29/25		
Development of Base RAG															10/06/25		
Embedding Pipeline															10/20/25		
Download Pipeline															10/20/25		
Data Storage Research															10/20/25		
Sprint 3															11/10/25		
Develop Storage Solution															10/27/25		
Finish Results Page															10/27/25		
Implement Langchain Toolkit															11/10/25		
Sprint 4															11/17/25		
Subsystem Validation															11/21/25		
Error Handling															11/17/25		
Final System Demos															11/24/25		

Validation Plan for The Patent Miner

Test Name	Success Criteria	Methodology	Status	Responsible Engineer(s)
API/Pydantic Test	API gives correct response depending on the route taken 100% of the time	Send input query and data of different variations including edge cases like empty queries, queries with special characters, and get	Tested	CJ Vines
Determinism Test	Preprocessing Subsystem resulting chunks consistent across runs on the same dataset	Calculate SHA-256 hash of entire list of dictionaries after chunking and compare hash across runs to ensure download pipeline is deterministic. If any run is inconsistent, the test fails	Tested	CJ Vines
Mock End-To-End Test	Query on Frontend gets correct response through API, displays patent chunks as well as metadata	Reconstruct patents from snippets after chunking and build a mock retrieval and RAG pipeline, run 20 queries and display results	Tested	CJ Vines
Metadata Extraction	Stored metadata is verified to be at least 99% accurate	Compare metadata after chunking to metadata called from USPTO API, aiming for 99% accuracy	Tested	CJ Vines
Embedding Reranking Test	"Golden" query - chunk pairs must have $\geq .05$ Recall@k, Mean Reciprocal Rank (MRR) $\geq .5$, and Normalized Discounted Cumulative Gain (NDCG) of .5	k = 10 target chunks at minimum with an equal number of matching queries. Calculate Recall@k, MRR, and NDCG for dataset	Tested	Christian Casteel
Embedding Accuracy Test	For a set of chunks, Hit@k of 1 and precision at k above .5. Relevance determined by patent id	Query containing information from multiple patents with similar CPC codes will test retrieval. Calculate Hit@k and Precision@k on top k chunks where k = 10	Tested	Christian Casteel
Failure Compliance Test	Pipeline can take in incorrect patent data and continue embedding without failure	Patent containing chunks exceeding token limit, empty sections, and an incorrect CPC code will be sent through the pipeline. Success will be determined by the patent traveling through the pipeline, and the false data being flagged and viewable	Tested	Christian Casteel
Metadata Integrity Test	All stored metadata must be 100% consistent with metadata listed on a patent	Embed 5 patents, specifically patents with unique CPCs and numbers of authors, then compare all metadata fields to the same patents on the USPTO database	Tested	Christian Casteel
Full Pipeline Stress Test	USPTO bulk dataset is downloaded, embedded, and stored without failure	USPTO bulk dataset download is conducted via CJ's download script. The bulk dataset should contain at minimum 100 patents, and success is determined by storage process completing without failure	Tested	Christian Casteel
Query Metadata Extraction	Extract all relevant metadata from user query, and use the metadata to filter patent search	Execute patent search using queries containing various metadata including prior date, patent ID, CPC etc.	Tested	Kosi Ezewudo
Query Cleaning	Removes typos, sentence fragments etc. from user query	Execute patent search using grammatically incorrect queries	Tested	Kosi Ezewudo
Generation Accuracy and Quality Test	Generated responses must have $\geq .8$ Answer relevancy scores, $\geq .8$ Diversity and Novelty	Query RAG pipeline using golden set of patents. Generate responses and calculate Answer relevancy, Diversity and Novelty scores	Tested	Kosi Ezewudo

**THE PATENT MINER
CJ VINES
CHRISTIAN CASTEEL
KOSI EZEWUDO**

403 SUBSYSTEM REPORTS

REVISION – 2
27 April 2026

SUBSYSTEM REPORTS FOR THE PATENT MINER

APPROVED BY:

CJ Vines

12/06/25

Prof. Nowka

12/06/25

Fahrettin Ay

12/06/25

Table of Contents

1. Preprocessing Subsystem Report.....	47
1.1. Subsystem Introduction	47
1.2. Subsystem Details.....	47
1.3. Subsystem Validation	48
1.3.1. Validation Preface	48
1.3.2. Deterministic Hashing.....	48
1.3.3. Metadata Completeness.....	49
1.4. Validation Conclusion	50
2. Embedding and Storage Subsystem Report	51
2.1. Subsystem Introduction	51
2.2. Subsystem Details.....	51
2.3. Subsystem Validation	52
2.3.1. Retrieval Accuracy	52
2.3.2. Pipeline Robustness.....	53
2.4. Subsystem Conclusion	53
3. Frontend Subsystem Report.....	54
3.1. Subsystem introduction	54
3.2. Subsystem Details.....	54
3.3. Subsystem Validation	54
3.3.1. Mock End-to-End Validation	55
3.3.2. API Validation.....	57
3.4. Subsystem Conclusion	57
4. Retrieval Augmented Generation (RAG) Subsystem Report.....	58
4.1. Subsystem Introduction	58
4.2. Subsystem Details.....	58
4.3. Subsystem Validation	59
4.3.1. Query Cleaning Layer.....	59
4.3.2. Query Metadata Extraction Layer	60
4.3.3. Generation-metrics	60
4.4. Subsystem Conclusion	60

List of Figures

Figure 1. Output of Hash Calculations.....	46
Figure 2. Bar Chart Indicating Successes and Failures.....	47
Figure 3. Subsystem Flow Chart.....	49
Figure 4. Retrieval Metrics.....	50
Figure 5. Faux Rag Answer with Citations.....	53
Figure 6. Figure Thumbnails.....	54
Figure 7. Metadata Displayed to User.....	54
Figure 8. Console Output of Tests.....	55
Figure 9. RAG Pipeline.....	57
Figure 10. Analysis on RAG Performance on Query Metadata Extraction.....	58

1. Preprocessing Subsystem Report

1.1. Subsystem Introduction

Any RAG pipeline is only as accurate as the data that it retrieves. The preprocessing subsystem aims to supply our embedding model with the most accurate data possible, so that downstream processes are not affected by poor data quality. The preprocessing subsystem was tested for accuracy of metadata and determinism in order to ensure that no extra data is added to any chunk or metadata, as well as verify the accuracy of parsing by comparing parsed metadata with a separate source of metadata. This process ensures all data parsed is accurate and backed by multiple sources.

1.2. Subsystem Details

The preprocessing subsystem is responsible for preparing patent text and metadata so that it can be used reliably by the embedding pipeline. Its primary objective is to generate clean, consistently formatted text segments while preserving information such as styling and structure that may be significant for interpretation. All processed content is organized by document ID and stored in a standardized format to support efficient indexing and retrieval.

Patent data is sourced from the USPTO Patent Grant Full Text Dataset with Embedded Images (PTGRDT), which serves as the authoritative reference for granted patents and is also used by external tools such as Google Patents. Because the patent XML files are already organized hierarchically, only minimal parsing logic was required. However, extracted text contains formatting artifacts that must be removed to ensure consistency. The preprocessing subsystem performs text cleaning to eliminate extraneous characters and produce readable, uniform chunks.

Rich Text formatting is used during preprocessing to preserve features such as bolding and underlining that may be referenced within patent documents. Prior to embedding, this formatted text is converted to plain text to ensure compatibility with the embedding model. The preprocessing pipeline supports Utility, Design, and Plant patents, including patents that contain genomic sequence information. Sequence data is processed separately but retained under the same document identifier as its associated patent to preserve contextual linkage.

To improve robustness, partially downloaded files are saved with temporary file extensions so that failed downloads can be identified and retried. The subsystem also verifies the number of patents in each dataset release and provides real-time progress feedback during execution. Patent text is currently chunked according to document structure, with oversized sections split into multiple chunks when token limits are exceeded. This chunking strategy is designed to be extendable, allowing for future incorporation of overlapping context when needed.

1.3. Subsystem Validation

1.3.1. Validation Preface

The preprocessing subsystem was initially validated for effectiveness through non-quantitative means. The original validation plan was twofold; reconstructing a random sample of patents in markdown format by concatenating each chunk to said document and comparing the output to the full text online. This validation methodology was also made easier by the validation methodology of the mock end-to-end test, which allowed for referenced patents in a mock RAG answer to have their metadata displayed to the user, which includes a hyperlink to the Google Patents page to the referenced patent. Although it looks good for a demo, these tests do not validate the completeness of the pipeline in parsing.

1.3.2. Deterministic Hashing

In order to rectify this lack, new validation criterion was introduced that validate both the completeness and determinism of the download pipeline itself. Enter deterministic hashing, a technique of validating determinism of a preprocessing system in data science. This is achieved by calculating the SHA-256 hash of the resulting output of the pipeline. The SHA-256 hash is a cryptographic algorithm that gives a unique 256-bit representation of the file based on the amount and position of bits that make up the file. If any bits are missing or out of order, the hash changes drastically. The preprocessing pipeline was executed on 5 different datasets offered by the USPTOs Bulk Dataset PTGRDT product 5 separate times, deleting the leftover file each time. After calculation of the SHA-256 hash it was found that the hash was the same each time, validating the determinism of the pipeline.

```
[determinism] PASS: 10 run(s); run_hash=9c526c42e3f9b1bd17b32d0e1d81220a4868e94023cdea329204f8c33021bc84
[determinism] PASS: 5 run(s); run_hash=0897747fd7f202c822a3211731103a60d73952cb055e7f55d767fc10732d789f
[determinism] PASS: 5 run(s); run_hash=b3ac665d2f1f86e04d3a5e63ab8a09a0f1c1003f98758ac9594d7750d91f2efb
[determinism] PASS: 5 run(s); run_hash=fa21f7ee04baf2f82b36fe860aba319426974b89fa84b9ea1a268498b05b4c4a
[determinism] PASS: 5 run(s); run_hash=4ddf3ba3a1fbc77543e150c06f60c65dc7e320f6cb2d6b54a20a0c7c8f9d7d27
```

Figure 1. Output of hash calculations

1.3.3. Metadata Completeness

The next validation is aimed to ensure that metadata extracted is verified for completeness. Complete metadata is crucial for proper retrieval filtering, as well as for the user's research purposes. In order to properly validate the metadata being extracted, cross source inspection is used. The USPTO File Wrapper API (Metadata API for short) is able to provide the metadata for any patent on demand, so long as the application number was provided. During chunking, when metadata is extracted the Metadata API is called and the resulting strings are compared, and any mismatches are logged to a CSV. This process was done across the same 5 datasets as the determinism test.

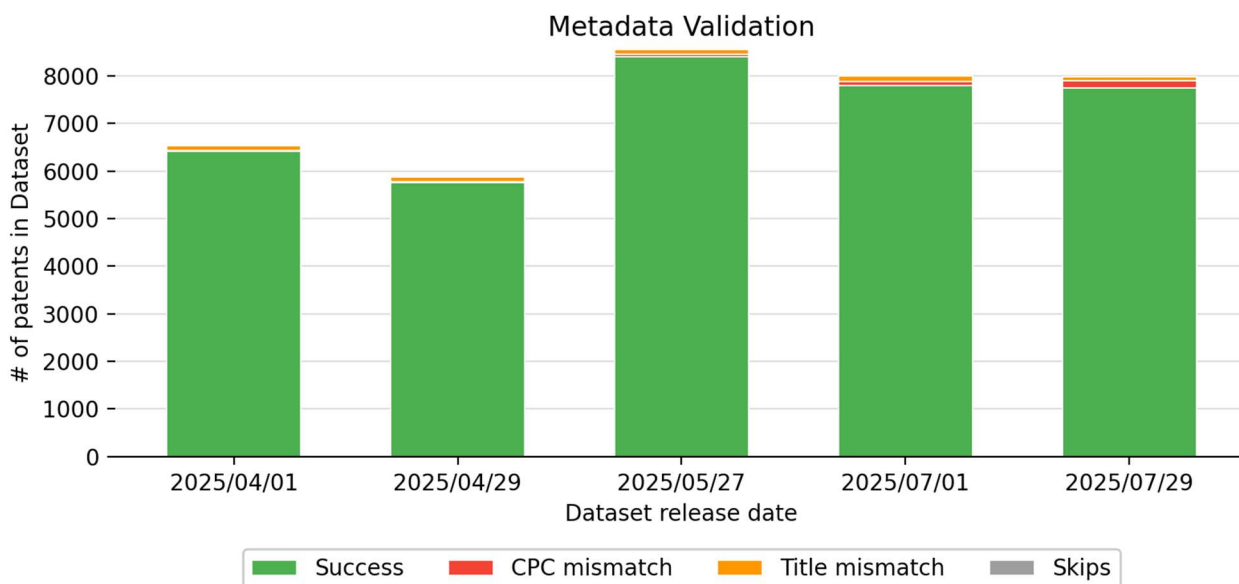


Figure 2. Bar chart indicating successes and failures

After chunking, unsuccessful matches were found to be a part of 3 categories, CPC mismatch, title mismatch, and skipped. About 98 percent of patents had successful metadata extraction.

Dataset	Success	CPC Error	Title Error	Skips	Total
2025/04/01	6423	16	90	0	6529
2025/04/29	5763	10	98	2	5872
2025/05/27	8414	45	105	0	8563
2025/07/01	7795	87	119	0	7998
2025/07/29	7751	144	90	0	7982
Total	36146	302	502	2	36944
Percentage	97.84	0.82	1.36	0.01	100

Table 1. Success rate of metadata extraction

Of the 2 types of mismatches that occurred, the most notable mismatch was the CPC mismatch. The CPCs that are in the XML are the same that are on the patent when it is filed. During the patent review process, more CPCs can be added to it by examiners. These changes are reflected in the Metadata API and could possibly have more up to date information. Of the title mismatches, all mismatches were either due to capitalization issues or inconsequential title changes like missing articles. 2 skips happened during testing, where the API was unreachable do to internet issues. To avoid excess API calls, the test for those 2 patents was not ran again.

1.4. Validation Conclusion

The quantitative tests along with the mock end-to-end tests fully validate the preprocessing subsystem. Target accuracy of patent data is achieved at 99.2 percent of all data extracted being validated as accurate. Further research will be done to try to identify signals which might indicate a patent could have had an updated CPC. The pipeline is also showed to be deterministic, and thus is a reliable method of downloading and chunking patent data, and ensures downstream processes get the most reliable information possible.

2. Embedding and Storage Subsystem Report

2.1. Subsystem Introduction

The embedding and storage subsystem is the core of The Patent Miner. Its primary function is to tokenize patent text chunks, convert these tokenized texts into vectors, and then store these embeddings into an index for later retrieval. This subsystem also takes in patent text and metadata and stores those properties alongside the vectors for filtered searching and identification for chunks. The system generates multivector embeddings using ColBERT and stores all patent data using a local Weaviate database. This provides a semantically meaningful database for interaction with the orchestrator and frontend, prepared for integration with a RAG pipeline.

2.2. Subsystem Details

A block diagram for the subsystem is shown below.

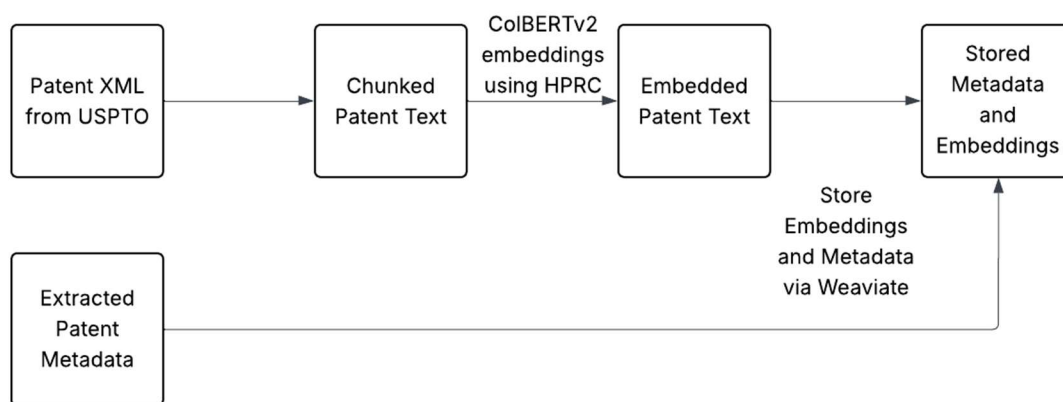


Figure 3. Subsystem Flow Chart

Patent metadata is extracted at the time of download and is stored with the associated patent chunks after embedding is complete. Patent chunks consist of text sections from the abstract, description, and claims. The subsystem can embed and store patent information directly from the USPTO bulk download and is seamlessly connected to the download portion of the frontend.

HuggingFace AutoTokenizer breaks chunked patent text into subwords, or tokens. This has a set limit of 350 tokens per chunk to ensure the ColBERTv2 token limit of 512 is not exceeded. ColBERTv2 interprets tokenized patent text and generates multivector embeddings containing semantic information. Weaviate compresses these multivector embeddings into single vectors and stores them locally alongside their associated metadata and original text. The result is a robust index with various metadata and semantic information for patents fully prepared for RAG.

The size of the database is limited by disk space on the operating machine. The database can be added to iteratively, and existing patent data does not have to be embedded again in the event of additional patents being downloaded.

2.3. Subsystem Validation

2.3.1. Retrieval Accuracy

The embedding and storage subsystem was tested and validated based on retrieval via Weaviate. The ColBERTv2 model, database, and chunking strategy were all validated as a combined unit. This accurately evaluates the entire pipeline's performance. Queries were embedded in the same method as patent text chunks. Similarity of embeddings were calculated and ranked directly by Weaviate using its built-in cosine similarity calculations. Cosine similarity is defined as:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

Where A and B are two compared vectors. The closer to 1, the more similar two vectors are, and thus the more semantically similar a query and chunk are. Weaviate uses this to rank the top k vectors, which all metrics were based on.

All tests were performed on a dataset of 6799 application patents, with 10 of those patents being the test set evaluated on. 20 test queries pertaining to the claims of the test set were created with the intent of returning chunks from relevant patents. A relevant chunk is any chunk belonging to a target patent. Metrics were evaluated on the top k patents, where k = 10.

Metrics include Precision@k, Recall@k, Hit@k, MRR, and NDCG@k. Precision@k is the number of relevant chunks out of the top k. Recall@k is the number of relevant chunks returned out of the total possible relevant in a patent. Hit@k is 1 if a relevant chunk is present in the top k, 0 otherwise. MRR is the 1 divided by the number rank of the relevant chunk. NDCG@k gives a score to each k ranking, the closer to the top relevant chunks are to the top, the closer to 1 the score is.

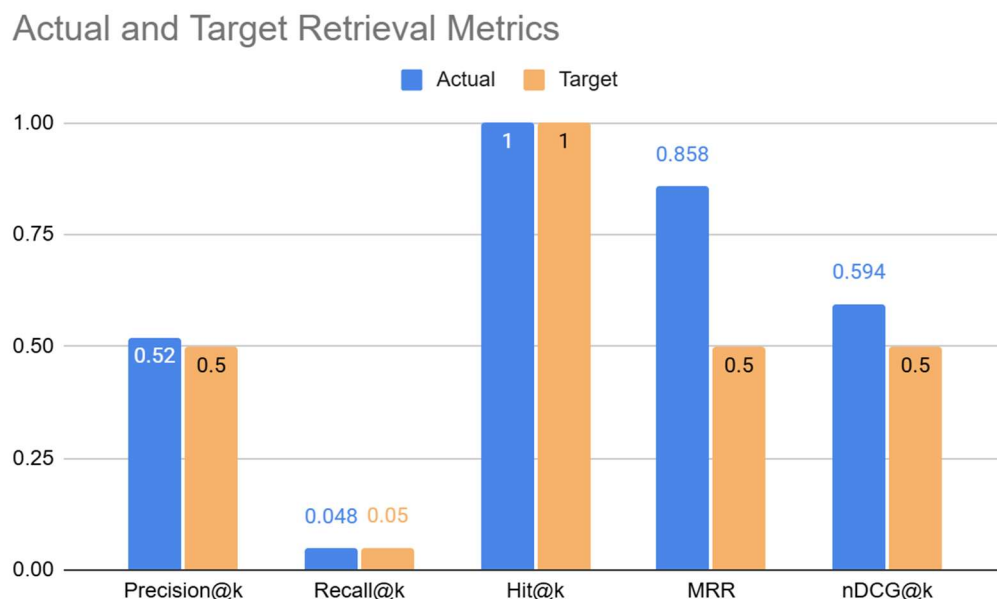


Figure 4. Retrieval Metrics

MRR far exceeded validation expectations. 0.858 means the first relevant patent most often appeared as the first result as opposed to the target 0.5, the second ranked result. Beyond that, all calculated metrics met their target value apart from Recall@k. On average patents contained 114 chunks, however some contained as many as 300. Outliers like these lower Recall@k, and thus slightly below target score is not concerning. This data means the subsystem has a high tendency to return relevant patents as the first result and validates the functionality of the subsystem.

2.3.2. Pipeline Robustness

The robustness of the subsystem was also tested using more qualitative methods. These tested generally included the pipeline's ability to continuously run and properly store data, even under large volume downloads. These tests include a failure compliance test, a metadata integrity test, and a download and upload stress test. All of these tests were evaluated prior to the retrieval accuracy test.

The subsystem could embed and store the 6799 patent dataset all in a single sitting. Embedding using the High Performance Research Computing took 2 hours and 40 minutes, and storage on my personal machine took 12 hours. During these processes chunk truncations and CPC issues occurred without halting the embedding process or the download process. Both processes executed without stopping under the 6799 patent load, which is the general volume of weekly patent bulk datasets.

The 10 patent dataset used in the retrieval accuracy validation was used to determine metadata integrity. The CPC, authors, priority data, section, and document id of these stored patents were compared to the original patent XML files, and all fields matched exactly.

2.4. Subsystem Conclusion

All aspects of the subsystem worked as expected. In some regards, namely MRR, the subsystem exceeded expectations. Currently, MaxSim reranking and semantic chunking are not implemented, meaning cosine similarity is the only measure of similarity, and truncated chunks are cut off abruptly. Despite this, the metrics are still strong. This leaves a large amount of room for improvement. The subsystem has already proven to be able to interface with the downloading subsystem and is fully prepared to integrate with the orchestrator. In doing so, it shall provide the orchestrator with accurate metadata and contextually relevant results.

3. Frontend Subsystem Report

3.1. Subsystem introduction

The Frontend serves as a user interface for entering queries for the Patent Mining Tool. The front page is a simple hero page with a search bar, and when a query is entered, the query is sent to an API, which will trigger one of two processes to occur. Then, a response is generated and sent back to the front end to display the results of the search to a user, with all relevant metadata included. This subsystem was designed to be simple and approachable for all users, while still giving accurate results in an aesthetically pleasing format.

3.2. Subsystem Details

The Frontend is built upon the React framework, utilizing Tailwind for CSS styling, and implements a FAST API backend. React was chosen because it is the industry standard for frontend development, and Tailwind was chosen for its ease of use in keeping styling consistent across elements. FAST API was chosen for ease of implementation, as well as to keep the language of the API similar to the backend.

When a query is entered the Frontend makes a GET request through the API. This request must contain all relevant information required by the post request as outlined in the ICD. Once all other subsystems have finished processing the query, the response expects a defined structure, and sends the relevant response, chunks, and metadata back to the front end through the API to be displayed to the user.

3.3. Subsystem Validation

Validating the Frontend was made easier by the prohibition of purchasing a domain by the department. This means that live tests of the webpage were not allowed. The only remaining validation that was required was to test the results page after requests have been made, and performing validation of the API under different combinations of inputs, and inducing bad response codes to ensure proper functioning of the API.

3.3.1. Mock End-to-End Validation

This methodology was very challenging to execute. After the preprocessing pipeline finishes chunking all patents in the dataset, a few patents that are randomly chosen to remain in memory to be combined back into the original document afterwards. These documents were chosen to be in the markdown format form to preserve rich text, a functional requirement of the pipeline to ensure no context is lost when the LLM is regenerating its response. Then, these documents are combined into a JSON to act as a fake database. Following that, a GET request, the query is searched for in the catalogue, and patents with the relevant terms are returned as the faux top-k retrieved patents. After, a fake LLM response is generated that cites the top-k patents, and those patents are displayed in the same order that they are sent in.

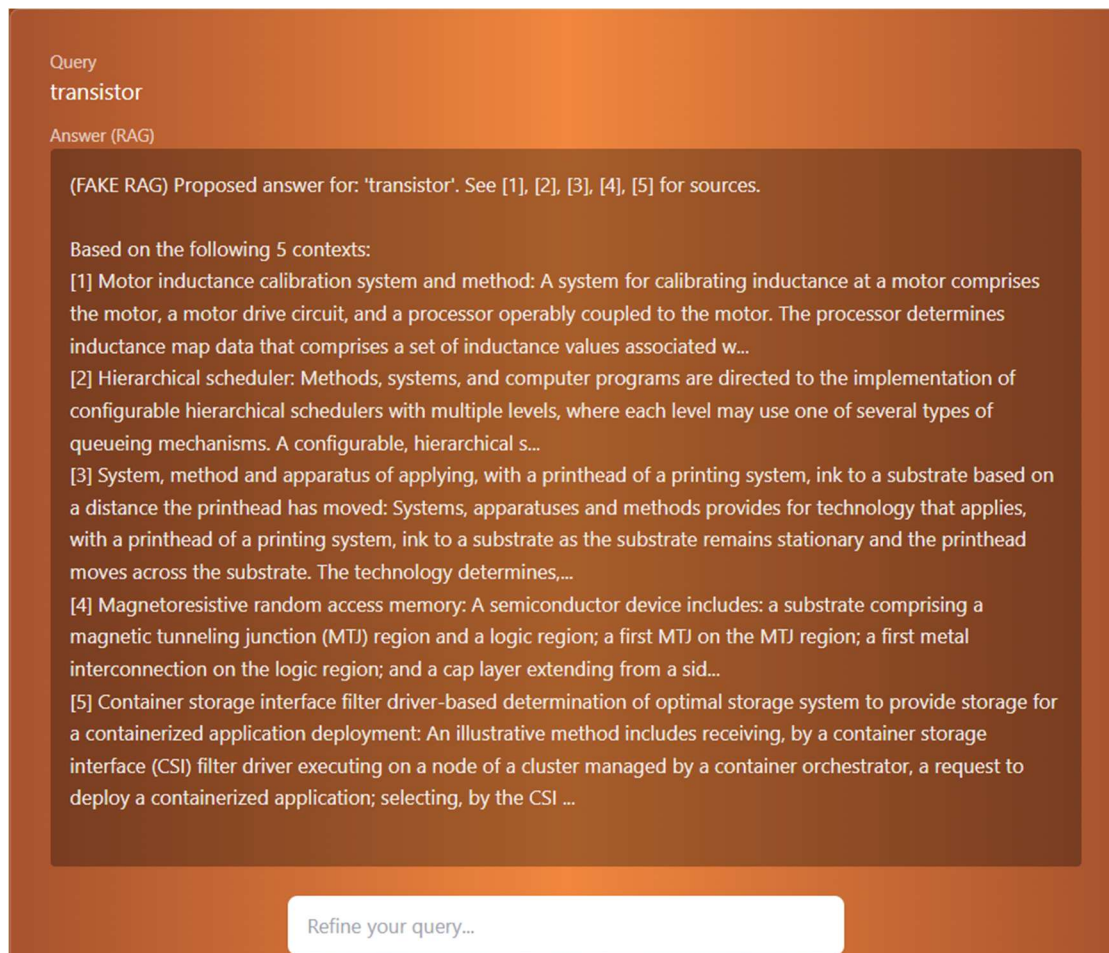


Figure 5. Faux RAG answer with citations

This method of validation was ran 10 times with different seeds to visually inspect patents from the dataset, and all text and metadata was found to remain in the sampled patents after completion. The other benefit of this validation is that all that needs to remain is to replace the fake subsystems with real subsystems, eliminating the need for more work in the future.

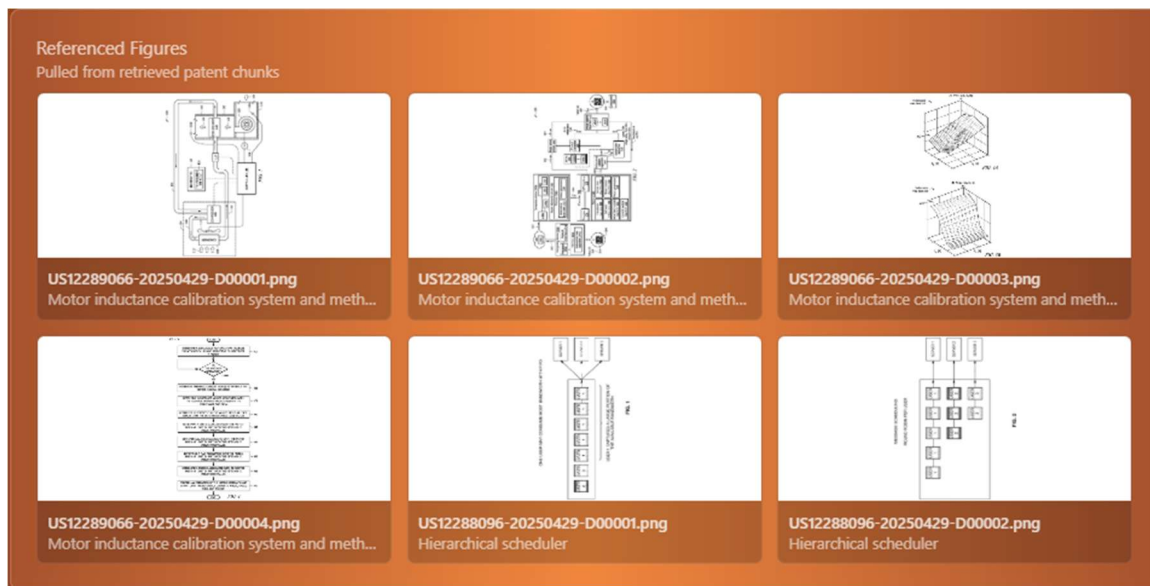


Figure 6. Figure thumbnails

Results

Cited Patents

[1] Motor inductance calibration system and method

A system for calibrating inductance at a motor comprises the motor, a motor drive circuit, and a processor operably coupled to the motor. The...

Title: Motor inductance calibration system and method

Authors: Palanivel Subramaniam; Yan Yang; Kishor Satish Gaikwad; Sunil K. Khiani; Stephen J. Vendt; Nasim Arbab Rahman

Classification: H02P 21/0017

Filing Date: 2023/07/05

Kind: B2

doc_id: 12289066

[Google Patents full text](#)

Figure 7. Metadata displayed to user

3.3.2.API Validation

Validating the API requires testing all routes correctly accept or reject inputs based on the predefined JSON schema for input and output. This includes testing the standard /healthz route that monitors the status of the API, as well as the GET route with RAG mode both enabled and disabled. For queries, special characters were used to ensure this did not cause errors in tunneling to the backend as well as on faux retrieval. Out of bounds k values were chosen to test whether this would be rejected by FAST API as well.

```
(backend-py3.13) PS C:\Patent_Wizard> pytest tests/test_api.py
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Patent_Wizard
configfile: pyproject.toml
plugins: anyio-4.10.0
collected 37 items

tests\test_api.py ..... [100%]
===== 37 passed in 5.75s =====
(backend-py3.13) PS C:\Patent_Wizard>
```

Figure 8. Console output of tests

All tests were passed after Pytest finished running, showing the only route in the API functions as expected, so no malformed JSONs missing information will cause errors in backend processes downstream, or issues displaying results to the user.

3.4. Subsystem Conclusion

All in all, Frontend validation was successful. The subsystem works as designed and fortunately the only elements that required tweaking was the aesthetics of the home page and results page. When all elements are combined, the Frontend enables the system to be used by many

4. Retrieval Augmented Generation (RAG) Subsystem Report

4.1. Subsystem Introduction

The RAG subsystem is the component of the Patent Miner primarily responsible for generating contextually relevant responses based on the user query and selected patents stored within the Patent Miner database. This component uses the query passed from the front-end User Interface to search the database containing embedded patents and generate responses using an LLM. Additionally, the RAG pipeline implements a quality check and aims to prioritize accuracy in retrieved patents during the database search and retrieval process by making use of a query cleaning layer, query metadata extraction layer and retrieved documents grading layer. This pipeline allows for a user-friendly interaction with Patent Miner.

4.2. Subsystem Details

The RAG pipeline is built upon the LangChain ecosystem and makes use of Google-gemini-2.5-flash. LangChain is chosen due to its widespread popularity and features provided for building LLM applications. Google-gemini-2.5-flash is chosen primarily for its capability to accept multi-modal data, allow for structured outputs and cost-effectiveness. The Patent Miner will specifically make use of the LangGraph orchestration framework within LangChain as it will allow for a more intricate and detailed flow of information through the RAG pipeline.

The RAG pipeline is primarily made up of 5 layers: i) Query Cleaning Layer ii) Query Metadata Extraction Layer iii) Patent Retrieval Layer iv) Retrieved Document Grading and Query Refinement Layer v) Generation Layer. As depicted in **Figure. 9**, after the user query is passed from the frontend interface – it is first analyzed within the query cleaning layer. In the Query cleaning layer, by the aid of an LLM as a judge, a binary check is run on the query to determine its grammatical quality – assigning a value of 1 or 0 to the query. If a score of 0 is assigned to the query, the query is cleaned, passed to the front-end for user validation, then passed back to the Query Metadata Extraction Layer. If a score of 1 is assigned to the query – the query is directly passed to the Query Metadata Extraction Layer.

Next, once the query reaches the metadata extraction layer, using an LLM again as a judge, the query is first examined to determine if patent context retrieval is necessary to answer the question or not. If not, the query is directly routed to the Response Generation Layer, however if patent context retrieval is necessary – any potential metadata contained within the query such as author name, filing date, CPC No. etc is extracted. The extracted metadata is stored in a structured format in preparation for the Patent Retrieval Layer.

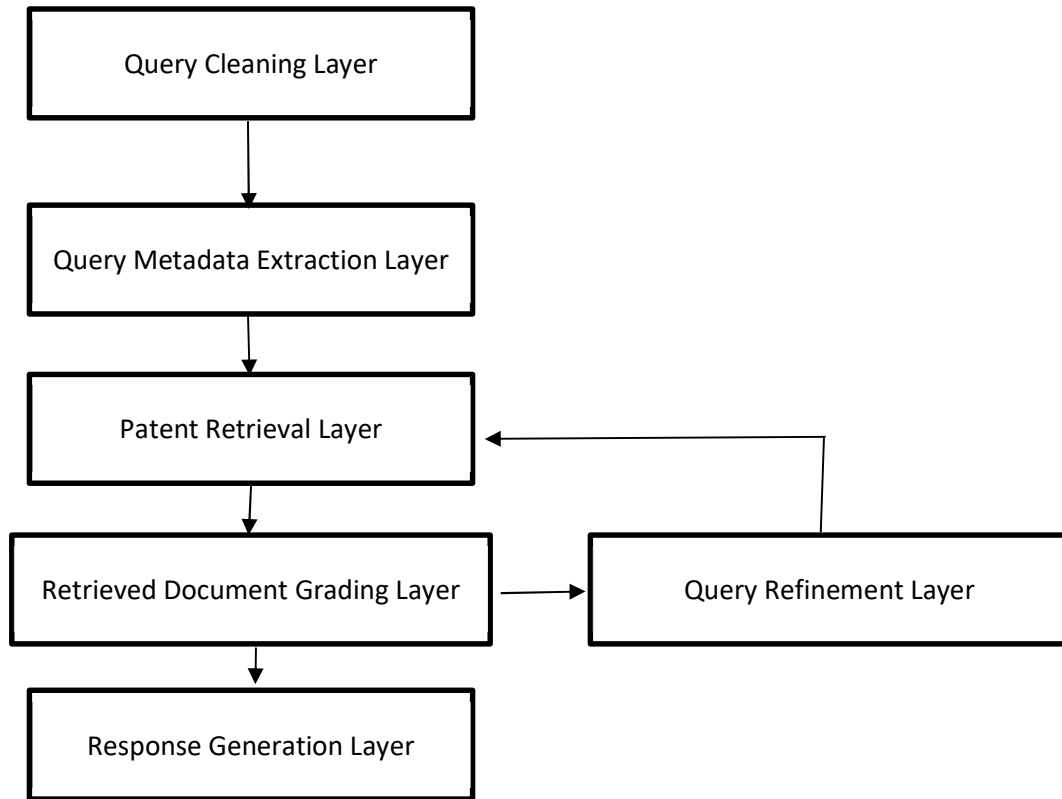


Figure 9. RAG Pipeline

In the Patent Retrieval Layer, the stored metadata along with the query is passed to the retriever and used to retrieve all patents relevant to the user query. After the relevant patents are retrieved, these are then passed to the Retrieved Document Grading and Query Refinement Layer. Like the Query Cleaning Layer, making use of an LLM-as-a-judge, a binary score is given to each retrieved document to ensure its relevance to the user query. If retrieved documents are irrelevant, then the user query is refined, and retrieval is re-executed again. If the retrieved documents are relevant, the “context” is then passed onto the response generation layer which generates the final response and passes it back to the frontend.

4.3. Subsystem Validation

For validation, 25 patents were selected from the USPTO database to validate the RAG system. To allow for full pipeline validation, Chroma database is used and GPT4All Embeddings is used.

4.3.1. Query Cleaning Layer

To validate the query cleaning layer, of the 25 selected patents, 15 patents are selected, and 15 unclean queries are made related to those patents. The RAG system is tested on its ability to detect the lack of grammatical quality in each query. Of the **15** unclean queries, the RAG pipeline detects the lack of grammatical quality in **all 15** questions and effectively cleaned each query.

4.3.2. Query Metadata Extraction Layer

To validate the query metadata extraction layer, 25 metadata-rich queries are crafted, and the Metadata extraction Layer is tested on its effectiveness to fully extract all metadata from queries. This layer is validated apart from the full system. All metadata is extracted from all 25 queries except two as shown in the chart below. The Query Metadata Extraction Layer is found to be robust to queries containing one metadata or a simple combination of metadata

4.3.3. Generation-metrics

The generation quality of the RAG system is evaluated against three metrics: i) Diversity ii) Novelty iii) Answer relevancy. Of the 25 patents, 15 patents are selected, and 15 good queries are crafted relating to each patent. The RAG pipeline achieves a Diversity score of 0.86, a Novelty score of 0.6, and an Answer relevance score of 0.59. The Diversity score of generated responses meets the success criteria of our validation plan. The novelty score and answer relevance scores do not meet the success criteria of our validation plan. However, the low novelty score is explained for by responses wherein the RAG system gives a response of “I do not know” as the novelty score metric used does not account for this edge case. Next, for the answer relevance score, the RAG system effectively addressed all queries during an eye-inspection of its responses, the relevance score of 0.59 is explained for by the fact that the relevance metric used does not account for an edge case of when the output is not yielded in a structured format.

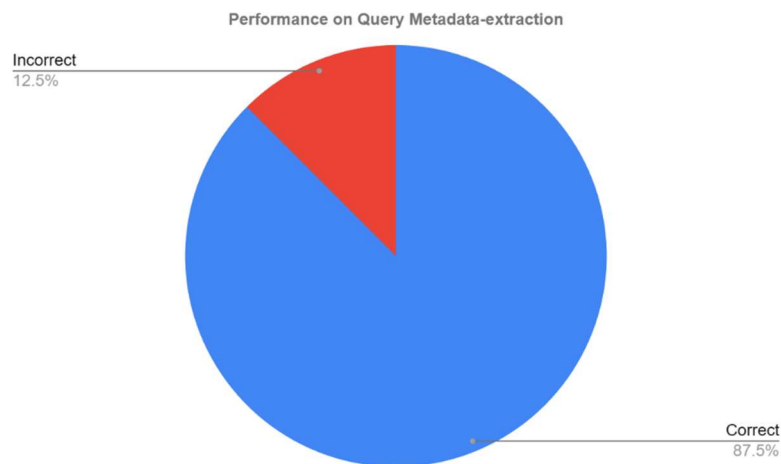


Figure 10. Analysis on RAG Performance on Query Metadata Extraction

4.4. Subsystem Conclusion

In conclusion, the RAG system tests were successful for the intended design. To prepare the RAG sub-system for a successful integration with the prior subsystems, future work will primarily include making the query metadata extraction layer robust to extracting complex metadata-rich queries and ensuring that the outputs of the generation layer will be formatted to adhere to the required frontend JSON schema.

THE PATENT MINER
CJ VINES
CHRISTIAN CASTEEL
KOSI EZEWUDO

ECEN 404 FINAL SYSTEM REPORT

GITHUB REPOSITORY:

https://github.com/CJVines56/ECEN404-2026SP-Patent_Miner

REVISION – 2
27 April 2026

SUBSYSTEM REPORTS FOR THE PATENT MINER

APPROVED BY:

CJ Vines

04/25/26

Contents

1. Introduction.....	65
2. ECEN 404 System Changes	65
3. System Integration.....	66
3.1. Introduction.....	66
3.2. Frontend-Backend/Orchestrator Integration	66
3.3. Orchestrator-Backend Integration.....	66
4. Integrated System Validation.....	67
4.1. Introduction.....	67
4.2. Orchestrator Validation	67
4.3. Retrieval Accuracy.....	69
4.4. Execution Plan	70
4.5. Validation Plan	71
5. Limitations and Future Work	71
5.1. Introduction.....	71
5.2. Database Scaling	71
5.3. Embedding Future Work.....	72
5.4. Orchestrator Future Work.....	72

List of Tables and Figures

Table 1. Orchestrator Results.....67

Figure 1. Retrieval Latency Results.....67

Figure 2. Trace Latency Results.....68

Table 2. Retrieval Metrics by Search Type.....69

Table 3. Retrieval Metrics by Patent Count.....69

Table 4. Validation Results Per Dataset.....70

Figure 3. Execution Plan.....70

Figure 4. Validation Plan.....71

1. Introduction

Patent Miner is a Retrieval Augmented (RAG) System with an embedded database intended to serve as a patent search tool for patent litigation professionals and researchers. Patent Miner is composed of three subsystems: i) The embedding pipeline ii) The Orchestrator iii) The Frontend subsystem. The embedding pipeline preprocesses patents from the USPTO (United States Patent and Trademark Office), embeds patent data as vectors and performs retrieval of relevant patents. The Orchestrator interprets user queries, embeds these queries and uses the retrieved patent context to generate relevant answers. The frontend/API subsystem provides a UI and connects the webpage to the backend.

Tasks in ECEN 404 included integration, database scaling, accuracy improvements, Orchestrator tuning, and validation. This report details all these tasks and details the limitations and future work for Patent Miner.

2. ECEN 404 System Modifications

2.1. *Embedding & Retrieval Pipeline*

BM25 keyword search was added to the retrieval pipeline to give Patent Miner a hybrid retrieval system. Hybrid search is considered industry standard for retrieval systems, and the additional keyword search helped with pulling patents given more exact wording, which ended up being crucial in the single success from Dr. Reddy's dataset.

Computing was moved to the Olympus cluster as opposed to a local machine. The larger the database became, the more CPU and GPU intensive each search became, hybrid search exacerbates this issue by forcing two retrievals and their associated calculations to operate in RAM at once. Moving to the Olympus cluster solved all computing issues.

2.2. *Frontend*

RAG vs direct search mode was removed in favor of patent and claim search mode. The RAG response only enhances the search experience, while having multiple search avenues made more sense for a mode of operations.

2.3. *Orchestrator*

The Orchestrator pipeline is re-designed in ECEN-404. The query metadata filtering, retrieved document grading and query refinement layers are removed. A general response, query routing and summarization layers are added. The query metadata filtering layer involved the orchestrator probabilistically extracting filters for retrieval given a user's query. The possible unreliability of the query metadata filtering layer was abandoned in favor of a drop-down filter menu on the frontend to allow for increased accuracy in retrieved results and ease of use.

A general response layer is added to allow for having a regular back and forth conversation with the Patent Miner based on patent-related or general topics. A query routing layer is added to allow for the Patent Miner to route between given a general response i.e no context retrieval required and executing retrieval to give a response. Lastly, a summarization layer is added to equip the Patent Miner with a short-term memory. This will allow for a running conversation between a user and the patent miner per-session.

3. System Integration

3.1. Introduction

The three subsystems of Patent Miner have well defined boundaries and connection points in addition to mockups of all other subsystems before integration. The focus for integration was ensuring data reliably transferred between different subsystems. The API is part of the frontend subsystem but is in the backend. By this design, the front-end was already capable of interfacing with the backend prior to integration.

A search in the final integrated system involves a user inputting a query and filters, and that information being sent to FAST API. Once Pydantic verifies all required fields, filter information is sent to the backend for retrieval, and query text is sent to the orchestrator for processing. The orchestrator RAG response and backend results are then both routed through FAST API back to the front-end where results are posted.

3.2. Frontend-Backend/Orchestrator Integration

Frontend-backend/orchestration was all done through the FAST API search endpoint. React collects query, mode, and filter inputs from the user and sends this information to FAST API. FAST API then serves as a middleman relaying information between the frontend and backend/orchestrator subsystems.

Due to the API being part of the frontend subsystem and fully validated, integration involved very few architecture changes. The focus was to ensure all frontend fields properly transferred to the backend/orchestrator and that returned data could properly be interpreted and displayed by the frontend. This includes user query, mode, and filter inputs sent to the API properly routing to the backend and orchestrator, and backend results and RAG responses properly routing to the frontend and displaying.

No issues were encountered during this portion of integration. The frontend was already verified to interface with the API, and the backend/orchestrator both had input ports during subsystem developed that matched the expected structure. Small changes to the output structure of both the backend and orchestrator subsystems were all that was necessary to route their results to the frontend.

3.3. Orchestrator-Backend Integration

The orchestrator and backend interact directly with one another. The orchestrator submits processed queries to the backend for retrieval, and those results are sent back to the orchestrator to be used as context for the RAG response.

Both subsystems communicate with internal functions established prior to integration. The query from the orchestrator is directly sent to Weaviate, where a search is conducted using the query information and user submitted filters. The same search results posted on the React frontend are also passed to the orchestrator to use as context.

No issues were encountered during this portion of integration. What were originally mockup inputs for the orchestrator were seamlessly replaced with true data. The backend functions as normal, receiving queries and outputting patent results.

4. Integrated System Validation

4.1. Introduction

To perform an end-to-end integrated system validation, example patents provided by Dr. Reddy illustrated successful cases in which vague claims were successfully invalidated by patent litigation specialists. The ultimate goal was to be able to use the vague claim as a query and find the more detailed claim during retrieval. Also, a hand labeled set of queries mapped to patents was used to test semantic based retrieval. To test the response generated by the orchestrator, a faithfulness test was performed to see whether the question asked by the user was properly answered with context.

4.2. Orchestrator Validation

The orchestrator was validated based on two primary criteria: i) Faithfulness ii) Task Latency. The two criteria examined enabled for a qualitative measure of the Patent Miner. The Faithfulness metric measures how factually consistent a response is with the retrieved context. Task Latency primarily measures the duration of time the Patent Miner takes to perform a given task. To examine the task latency of the Patent Miner – two tasks are reported: i) Retrieval latency ii) Trace Latency. Retrieval Latency refers to the duration of time taken to perform retrieval and Trace Latency refers to the duration of time taken for the final response to be generated given a single query i.e including retrieval

The validation set is made up of 40 random queries based on patents within the Patent Miner's Weaviate database. To enable the automation of validation tests - The Retrieval Augmented Generation Assessment Suite (RAGAS) and LangSmith platform is used. The table below reports the results from a single validation round.

Context Basis	Query Set	Faithfulness	Retrieval Latency (Sec)	Latency (Sec) 99 th Percentile
Claims	40	0.6511	60-80	250-300

Table 1. Orchestrator Results

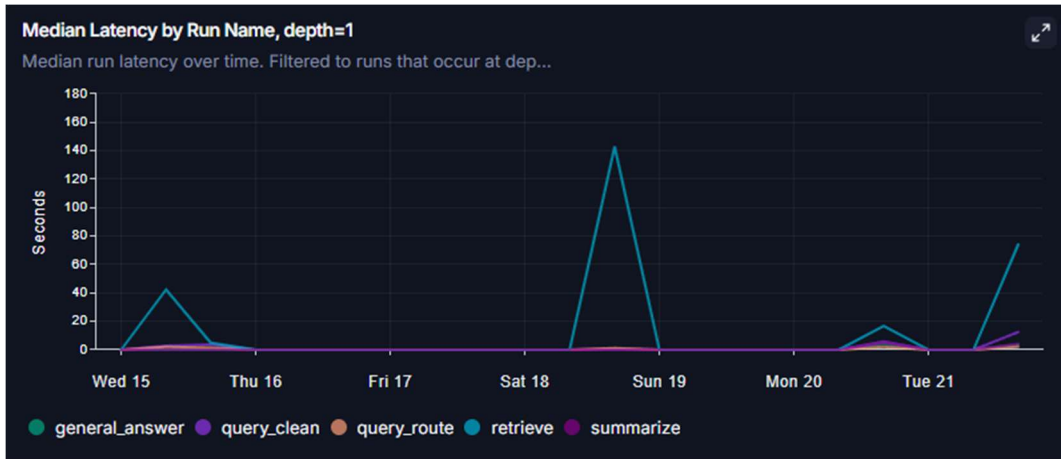


Figure 1. Retrieval Latency Results

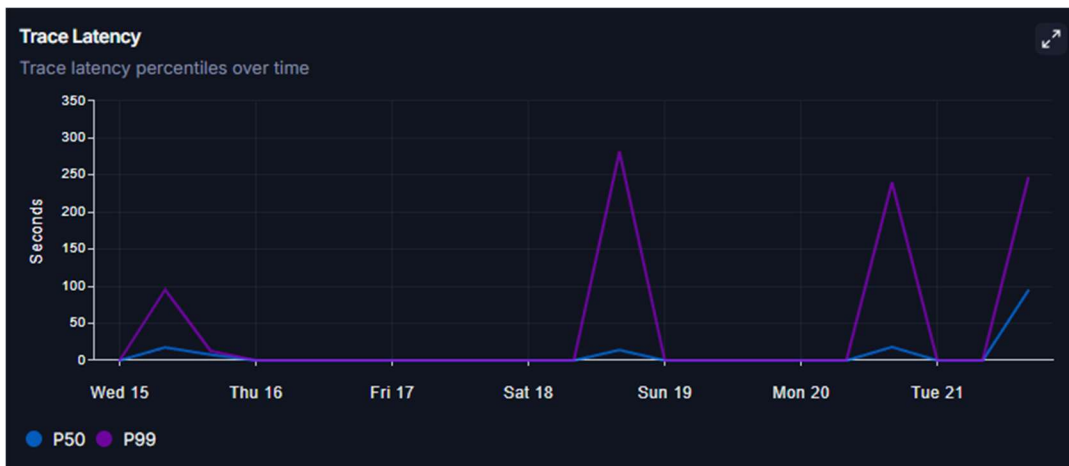


Figure 2. Trace Latency Results

From Table 1, a Faithfulness value of 0.6511 is obtained – this is below a satisfactory value. The intended users of our system are Patent Litigation professionals and researchers to whom accuracy is of high value. A potential solution to this is fine-grained prompt engineering aimed at reducing the risk of hallucination by the Large Language Model (LLM) used for response generation.

The Retrieval Latency of the Patent Miner resulted in 60-80 seconds, and the Trace Latency resulted in 250-300 seconds. A closer inspection into the retrieval latency chart indicates that the retrieval layer took the longest amount of time as other tasks took roughly 0 – 20 seconds. This can be explained by the size of our database which contained 1 million plus patents, thus extending the amount of time taken to search through the whole database for a single patent.

4.3. Retrieval Accuracy

Retrieval accuracy was validated using two methodologies. The first one being a more quantitative validation better suited for testing its broad research capabilities. The second method used the cases provided by Dr. Reddy, which better validated the scope of the project and its capabilities as a litigation tool. The first validation methodology proves to be the most important, as it tests Patent Miner's ability to provide accurate results, which is the core of the project. The secondary Dr. Reddy validation set is much more thorough, and was introduced later in the project timeline.

Validation method 1 specifically involved a set of 50 hand-labeled query-patent pairs, and evaluating average recall@k, nDCG@k, and MRR@k across the set. Query-patent pairs were designed to specifically test hybrid retrieval. 23 pairs contained exact keywords from target patents to verify the performance of BM25, 23 contained semantic language without keywords to stimulate MUVIRA, and the remaining 4 were highly vague queries to stress test both retrieval systems.

run	precision@10	recall@10	ndcg@10	mrr@10	candidate_hit@400
bm25	0.09	0.9	0.643757644	0.559444444	1
vector	0.0325	0.325	0.27693311	0.261458333	0.325
hybrid_a02	0.09	0.9	0.643757644	0.559444444	1
hybrid_a05	0.09	0.9	0.644853238	0.560694444	1
hybrid_a08	0.075	0.75	0.579690469	0.523571429	0.975

Table 2. Retrieval Metrics by Search Type

We performed numerous sweeps with the 50 pair set and determined that a 50/50 split between BM25 keyword search and MUVIRA vector search posed the best results. Going forward the 50/50 BM25 and MUVIRA retrieval paired with ColBERT reranking would be the setup used for validation.

Top Results (k)	Patent Count	Recall@k	nDCG@k	MRR@k
10	~50,000	0.9	0.645	0.561
10	~100,000	0.8	0.589	0.520
10	~1,000,000	0.825	0.691	0.650

Table 3. Retrieval Metrics by Patent Count

As the database scaled up, accuracy metrics dropped and had to be remedied with a MUVIRA vector dimensionality increase. This kept metrics above the target, although led to a hit to overall scalability and database size. This issue will be detailed further in the limitations portion of the report.

Validation method 2 included 7 sets of patents and their prior art selected by Dr. Reddy. This validation involved searching the exact claim 1 text from the original patent and retrieving all listed prior art patents in the top ten results. Patents used and results are listed below:

	Patent ID	Designation	Patent Type	Retrieved Prior Art?
Set 1	9485176	Original	Grant	N/A
	20040028064	Prior Art	Application	No
	20060072543	Prior Art	Application	No
Set 2	10735320	Original	Grant	N/A
	7477657	Prior Art	Grant	Yes
Set 3	8780887	Original	Grant	N/A
	20070157303	Prior Art	Application	No
Set 4	10020961	Original	Grant	N/A
	20150146727	Prior Art	Application	No
Set 5	10630505	Original	Grant	N/A
	10177957	Prior Art	Grant	No
	6502135	Prior Art	Grant	No
	20140359704	Prior Art	Application	No
Set 6	11108595	Original	Grant	N/A
	20100153558	Prior Art	Application	No
Set 7	9762397	Original	Grant	N/A
	7058973	Prior Art	Grant	No

Table 4. Validation results per Dataset

Validation on the Dr. Reddy patent set had unsatisfactory results. Only one out of the seven cases successfully returned patent prior art in the top ten results given the original patent claim 1. The limitations portion of this report will go into further detail as to why Patent Miner's retrieval system could not achieve the desired results in this specific validation.

Overall, Patent Miner exceeded initially established quantitative validation expectations. It provides semantically accurate results when provided queries of varying levels of detail. If scaled, it will prove to be a functional research tool for searching patents.

4.4. Execution Plan

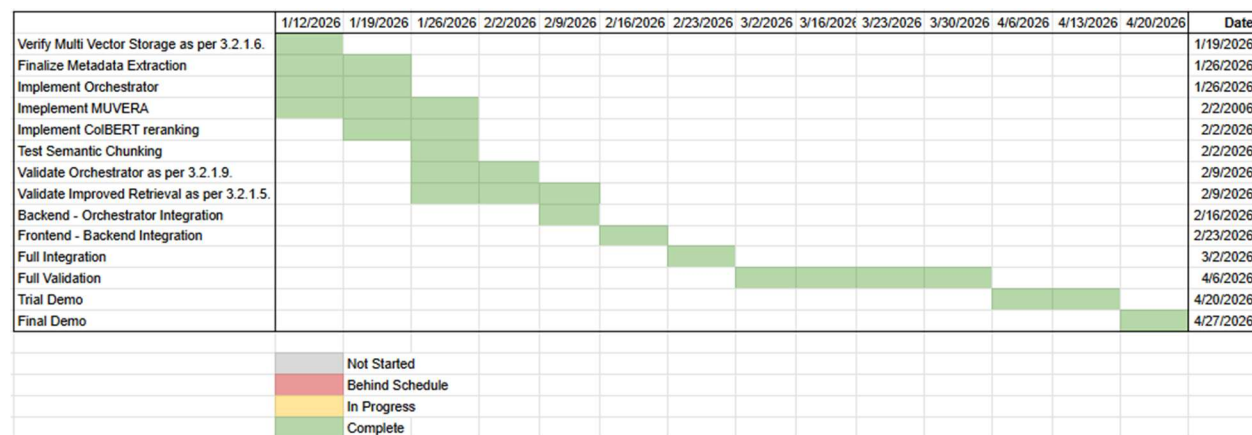


Figure 3. Execution Plan

4.5. Validation Plan

Paragraph #	Test Name	Success Criteria	Methodology	Status	Responsible Engineer(s)
3.2.1.2.	API/Pydantic Test	API gives correct response depending on the route taken 100% of the time	Send input query and data of different variations including edge cases like empty queries, queries with special characters, and get requests of the wrong shape	Tested	CJ Vines
3.2.1.1.	Determinism Test	Preprocessing Subsystem resulting chunks consistent across runs on the same dataset	Calculate SHA-256 hash of entire list of dictionaries after chunking and compare hash across runs to ensure download pipeline is deterministic. If any run is inconsistent, the test fails	Tested	CJ Vines
3.2.1.1.	Metadata Extraction	Stored metadata is verified to be at least 99% accurate	Compare metadata after chunking to metadata called from USPTO API, aiming for 99% accuracy	Tested	CJ Vines
3.2.1.4.	Full System Stress Test	Multiple simultaneous API calls are made successfully no data loss within 1ms	Make multiple API calls with different filters and modes, handle each concurrently in order.	Tested	CJ Vines & Kosi Ezewudo & Christian Casteel
3.2.1.5.	Embedding Accuracy Test	"Golden" query - patent pairs must have $\geq .8$ Hit@k, Mean Reciprocal Rank (MRR) $\geq .4$, and Normalized Discounted Cumulative Gain (NDCG) of .4	k = 10 target patents at minimum with an equal number of matching queries. Calculate Hit@k, MRR, and NDCG for dataset. "Golden" set will include 50 hand selected patents	Tested	Christian Casteel
3.2.1.1.	Failure Compliance Test	Pipeline can take in incorrect patent data and continue embedding without failure. Also will flag patents with false data	5 fake patents containing empty and improperly formatted metadata will be sent through the pipeline, and will throw error flags. System should continue embedding without failure	Tested	Christian Casteel
3.2.1.6.	Metadata Integrity Test	All stored metadata must be 100% consistent with metadata listed on a patent	Select 50 patents, specifically patents with generally unique CPCs and numbers of authors, then compare all metadata fields to the same patents on the USPTO database	Tested	Christian Casteel
3.2.1.1.	Download and Embedding Pipeline Stress Test	A USPTO bulk dataset is downloaded, embedded, and stored without failure	A USPTO bulk dataset download is conducted via frontend download script. The bulk dataset should contain minimum 1000 patents	Tested	Christian Casteel & CJ Vines
3.2.1.3.	Query Cleaning Test	Removes typos, sentence fragments etc. from user query	Execute patent search using grammatically incorrect queries	Tested	Kosi Ezewudo
3.2.1.9.	Generation Quality and Latency Test	Generated responses must have $\geq .8$ Faithfulness score and benchmark Latency results	Query Patent Miner using golden set of patents. Generate responses and calculate Faithfulness and Response Latency	Tested	Kosi Ezewudo & Christian Casteel & CJ Vines

Figure 4. Validation Plan

5. Limitations and Future Work

5.1. Introduction

The Patent Miner was limited in two primary aspects, namely; i) Storage Capacity ii) Quality of retrieval. To expand the database of downloaded patents, a high storage capacity was required for vector storage, and even more storage capacity would be required to include granted patents from earlier years, as well as application patents. Next, the retrieval mechanism is built on Weaviate which offered less opportunities for parallelization and control over the retrieval process

As a future work, to meet a primary requirement of our sponsor – a claim reconstruction layer must be implemented in the Orchestrator to streamline the patent litigation process.

5.2. Database Scaling

The 920 GB of claims currently embedded into vectors accounted for only 3 years of the 25 years that exists on the USPTO bulk dataset API's grant patents xml section (code PTGRDT). If the number of claims per patent and the number of tokens per claim were to be consistent with the 3 years that were embedded, about ~6-7 TB of space would be needed in the current implementation of Patent Miner to expand to every single patent grant available, not including application patents, of which there are many more. This doesn't include MUVERA improvements that will be discussed below, or database replication and sharding to allow for quicker searches.

5.3. Embedding Future Work

Weaviate worked well as a proof of concept to show how a multivector embedding model could be used to improve a retrieval portion of a rag pipeline. However, it is not a very efficient database, as to utilize multiple cores to improve search, this requires replicating the same database “shard” or section multiple times so it can be used on a separate core each time. To remedy this issue, FAISS could be used in order to parallelize retrieval on different sections of the same shard without replicating the dataset. The downside to this however is that Weaviate comes with a built in way to filter the database by metadata, as well as coming implemented with GraphQL calls for easy ways to traverse the HNSW graph that maps where the vectors are in memory on the PC. Switching to FAISS increases the complexity of the system, but many gains could be made from it

The other benefit for switching to FAISS is to get more control over the MUVERA single dense vector. Currently, the MUVERA representation is not accessible through Weaviate. This means you can only set the hyperparameters of MUVERA without actually being able to reduce the dimensionality itself. Increasing the dimensions could allow for a more accurate representation of the ColBERT vectors while saving space, and FAISS would allow for this change to happen.

5.4. Orchestrator Future Work

The Orchestrator is made up of four primary layers namely; i) Query Cleaning Layer ii) Query Routing Layer iii) Response Generation Layer iv) Summarization Layer. These layers encapsulated all but one of the basic needs of our sponsor – Claim reconstruction. Claim reconstruction is aimed at reconstructing a target claim from two retrieved and earlier patented claims. The purpose of this layer is to streamline the patent lawsuit process by invalidating patent lawsuits/infringement cases.

In addition, the orchestrator can further be optimized through prompt engineering to yield optimal faithfulness results. The integration of the Claim reconstruction Layer and prompt engineering will allow the Patent Miner to fully meet its intended use-case.